GUIA DO ALUNO

Monitor de qualidade do ar com Arduino e ESP32

DHT11 · DHT22 · MQ-2 · MQ-135 · MH-Z19C — teste de cada sensor, projeto com todos juntos e envio dos dados para a Rede de Telemetria do Espaço CRIA

Placas: Arduino Uno → ESP32 • **5 sensores + 1 projeto** • cada sensor: teste no Uno, depois no ESP32 com o celular

Projeto final: monitor de ar no Uno (com o gateway USB) e no ESP32 publicando no painel da telemetria-cria

Pré-requisitos: LEDs, LED RGB, `millis()`, Monitor Serial e divisor de tensão

Espaço Cria — 2026

Sumário

Como este guia funciona

Segurança, alimentação e níveis de tensão

Preparando o computador

Parte 1 — DHT11 (módulo de 3 pinos)

Parte 2 — DHT22 / AM2302 (3 fios)

Parte 3 — MQ-2: gás inflamável e fumaça

Parte 4 — MQ-135: “ar viciado”

Parte 5 — MH-Z19C: CO_2 por infravermelho

Conversor A/D ADS1115: por que e como usar nesta aula

Parte 6 — Projeto: monitor de qualidade do ar

Problemas comuns

Desafios

Referências

Como este guia funciona

Vamos montar um **monitor de qualidade do ar**: um aparelho que mede temperatura, umidade, gases e o **CO_2** de uma sala e avisa quando é hora de abrir a janela. Antes de juntar tudo, cada sensor é testado **sozinho**, primeiro no **Arduino Uno** (mais simples, 5 V) e depois no **ESP32** (3,3 V, com Wi-Fi e página no celular). Só quando os cinco funcionam separados montamos o projeto completo — assim, se algo falhar, você sabe exatamente qual peça investigar.

Parte	Sensor	Mede	Princípio	Interface
1	DHT11 (módulo 3 pinos)	Temperatura e umidade	Elemento resistivo de umidade + termistor NTC	1 fio, protocolo próprio
2	DHT22 / AM2302 (3 fios)	Temperatura e umidade (mais preciso)	Polímero capacitivo + termistor	1 fio, protocolo próprio
3	MQ-2	Gás inflamável (GLP, propano, H_2) e fumaça	SnO_2 aquecido: resistência cai com o gás	Analógica (AO) + digital (DO)
4	MQ-135	"Ar viciado": NH_3, álcool, benzeno, fumaça	Idem, com outra dopagem	Analógica (AO) + digital (DO)
5	MH-Z19C	CO_2 de verdade, em ppm	Absorção de infravermelho (NDIR)	Serial (UART) 9600 baud
5½	ADS1115	Tensão, com 16 bits (MQ pelo A/D externo)	Conversor sigma-delta + PGA	I2C
6	Projeto	Tudo junto + LED RGB de alerta	—	Uno: USB/gateway · ESP32: Wi-Fi/MQTT

Onde estão os arquivos

Códigos em `aula-ar/codigo/`: `NN_sensor_uno` (Arduino), `NNb_sensor_esp32_web` (ESP32 + celular) e `03c_mq2_esp32_ads1115_web` (MQ-2 pelo ADS1115). Projeto: `06_monitor_ar_uno` e `06b_monitor_ar_esp32_telemetria`.

Esquemas do Fritzing em `aula-ar/esquemas/saida/` (`.fzz` para abrir e editar no Fritzing, `.png` para imprimir).

Todos os sketches usam o **Monitor Serial a 115200 baud**. Os testes também funcionam no **Plotter Serial** (os valores saem no formato `nome:valor`).

Segurança, alimentação e níveis de tensão

- **ESP32 é 3,3 V.** Nenhum pino do ESP32 aguenta 5 V. Sinais que saem a 5 V (o AO dos MQ, o DO dos MQ) precisam de **divisor de tensão 1 kΩ / 2 kΩ** ou não podem ser ligados.
- **Os MQ esquentam.** O aquecedor interno puxa cerca de 150 mA e a carcaça passa de 50 °C. Não encoste e não deixe papel em cima.
- **Teste de gás sem chama:** use **álcool 70 % num algodão** a 10 cm. **Não use isqueiro de botão (piezo): ao apertar ele acende**, e o sensor está a 300 °C. Só o professor pode demonstrar com isqueiro de **pedra (roda)**, apertando a alavanca do gás **sem girar a roda**. Faça os testes com a janela aberta.
- **Desligue a USB antes de mexer nos fios.** VCC e GND invertidos matam um DHT em segundos.

- **Leia a serigrafia (o texto impresso na placa).** A ordem dos pinos muda de fabricante para fabricante; o guia mostra a ordem mais comum, mas quem manda é o que está escrito na sua peça.

Quanto de corrente o projeto pede

Peça	Tensão	Corrente típica	Pico
MQ-2 (aquecedor)	5 V	~150 mA	~180 mA ao ligar
MQ-135 (aquecedor)	5 V	~150 mA	~180 mA ao ligar
MH-Z19C	5,0 V ± 0,1 V	< 40 mA	125 mA (lâmpada IR)
DHT11 + DHT22	3,3–5 V	~3 mA	—
Arduino Uno	5 V	~50 mA	—
ESP32 com Wi-Fi	5 V (VIN)	~100 mA	~250–350 mA ao transmitir
Total (projeto ESP32)		**~0,5 A**	**> 0,8 A**

Por que usar uma fonte externa no projeto completo

Uma porta USB de computador entrega 0,5 A. Os testes individuais cabem nela; o projeto com os dois MQ, o MH-Z19C e o Wi-Fi do ESP32 não. Sintomas de fonte fraca: o ESP32 reinicia sozinho ("brownout"), o CO_2 some, as leituras dos MQ oscilam. Use uma **fonte de 5 V / 2 A** nas trilhas + e − da protoboard e ligue o **GND da fonte ao GND da placa**.

Preparando o computador

Na Arduino IDE, abra **Ferramentas → Gerenciar Bibliotecas** e instale: **DHT sensor library** (Adafruit) — ela pede para instalar junto a **Adafruit Unified Sensor**, aceite. Os MQ e o MH-Z19C **não precisam de biblioteca**: fazemos as contas e o protocolo no próprio código, para entender o que acontece. A seção do ADS1115 usa a **Adafruit ADS1X15** (instale junto a Adafruit BusIO). No terminal (Linux, laboratório):

```
terminal
arduino-cli lib install "DHT sensor library" "Adafruit Unified Sensor" PubSubClient "Adafruit
ADS1X15"
arduino-cli compile --fqbn arduino:avr:uno      aula-ar/codigo/01_dht11_uno
arduino-cli compile --fqbn esp32:esp32:esp32    aula-ar/codigo/01b_dht11_esp32_web
arduino-cli upload -p /dev/ttyUSB0 --fqbn esp32:esp32:esp32 aula-ar/codigo/01b_dht11_esp32_web
```

Para o projeto final no ESP32 você vai precisar também da biblioteca **CriaTelemetria** (do projeto `telemetria-cria`) — o passo a passo está na Parte 6.

Parte 1 — DHT11 (módulo de 3 pinos)

O que é e para que serve

O **DHT11** mede **umidade relativa de 20 a 90 % (±5 %)** e **temperatura de 0 a 50 °C (±2 °C)**, no máximo uma leitura por segundo (usamos uma a cada 2 s). É barato e suficiente para saber se a sala está abafada ou úmida demais (mofo!). O seu é um **módulo**: o sensor azul vem soldado numa plaquinha com **3 pinos**, um **LED** que acende quando há energia e um **resistor de pull-up** já instalado — por isso não precisa de resistor externo.

A física: um resistor que "bebe" água e um termistor

Umidade: o DHT11 tem um **elemento resistivo**: duas trilhas metálicas cobertas por um material que absorve água do ar. A água absorvida libera íons que conduzem eletricidade: **mais umidade → resistência menor**. O chip mede essa resistência e a converte em **umidade relativa** — quanto vapor há no ar comparado ao máximo que o ar aguentaria naquela temperatura. **Temperatura:** um **termistor NTC**, um resistor de cerâmica cuja resistência **cai** quando esquenta. Um microcontrolador de 8 bits dentro do sensor junta tudo e manda **40 bits** pelo único fio de dados: 16 de umidade, 16 de temperatura e 8 de verificação (*checksum*).

Pinagem do seu módulo

Pino (na ordem mais comum)	Nome na placa	Liga em (Uno)	Liga em (ESP32)
Esquerda	+ / VCC	5V	3V3
Meio	DATA / S / OUT	pino 2	GPIO 4
Direita	− / GND	GND	GND

Confira antes de ligar

No seu módulo o **dado fica no meio**. Há módulos parecidos com outra ordem (por exemplo, o KY-015, em que o meio é o VCC). Leia o que está impresso ao lado de cada pino. Se o LED não acender ao ligar, desligue na hora e confira VCC e GND.

Montagem e teste no Arduino Uno

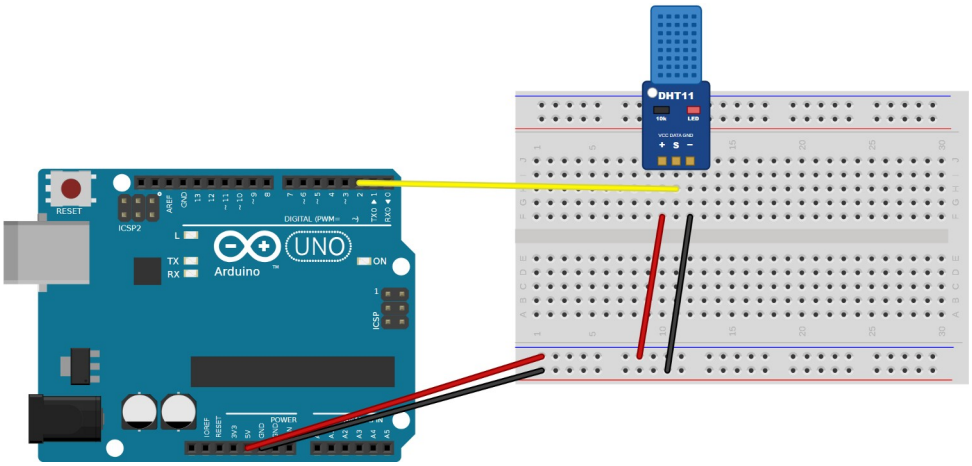


Figura 1 — Parte 1 — módulo DHT11 no Uno: + no 5 V, dados (meio) no pino 2, − no GND. O resistor de pull-up já está no módulo.

01_dht11_uno.ino

```
/*
  01 - Teste do MODULO DHT11 de 3 pinos (temperatura e umidade do ar)
  Placa: Arduino Uno/Nano     Bibliotecas: "DHT sensor library" (Adafruit) + "Adafruit Unified
Sensor"

  O seu modulo: placa com 3 pinos, LED de energia e resistor de pull-up ja soldado.
  Pinagem mais comum:   +  (VCC)  |  DATA / S / OUT  (MEIO)  |  -  (GND)
  CONFIRA A SERIGRAFIA: ha modulos com a ordem trocada. Nao precisa de resistor externo.

  Ligacoes:  + -> 5V    DATA (meio) -> pino 2    - -> GND
  Monitor Serial a 115200 baud. Tambem funciona no Plotter Serial (formato nome:valor).

  O que o teste confere:
   - o LED do modulo acende (energia chegando);
   - as leituras nao dao "falha" (fio de dados ok);
   - os valores sao plausiveis: sala tipica 24-32 C e 40-80 % (DHT11: +-2 C, +-5 %).
*/
#include <DHT.h>

const int PINO_DHT = 2;
const unsigned long INTERVALO_MS = 2000;   // o DHT11 so aceita 1 leitura a cada 1-2 s

DHT dht(PINO_DHT, DHT11);
unsigned long leituras = 0, falhas = 0;

void setup() {
  Serial.begin(115200);
  dht.begin();
  Serial.println(F("# Teste do DHT11 (modulo 3 pinos) - aguarde 2 s pela primeira leitura"));
}

void loop() {
  delay(INTERVALO_MS);
  float umidade = dht.readHumidity();
  float temperatura = dht.readTemperature();       // graus Celsius
  leituras++;

  if (isnan(umidade) || isnan(temperatura)) {      // NaN = a leitura nao fechou
    falhas++;
    Serial.print(F("# FALHA na leitura ("));
    Serial.print(falhas); Serial.print(F(" de ")); Serial.print(leituras);
    Serial.println(F(") - confira: fio DATA no pino 2? VCC e GND na ordem da placa? LED
aceso?"));
    return;
  }

  float sensacao = dht.computeHeatIndex(temperatura, umidade, false);   // indice de calor
  Serial.print(F("Temperatura_C:")); Serial.print(temperatura, 1);
  Serial.print(F(" Umidade_%:"));    Serial.print(umidade, 0);
  Serial.print(F(" Sensacao_C:"));   Serial.print(sensacao, 1);
  Serial.print(F(" Falhas:"));       Serial.println(falhas);
```

```
}
```

Entenda o código

- `DHT dht(2, DHT11)` diz à biblioteca em que pino está o sensor e qual modelo é.
- `readHumidity()` e `readTemperature()` devolvem **NaN** (*not a number*) quando a conversa com o sensor falha. `isnan()` detecta isso — e o sketch conta as falhas.
- `computeHeatIndex()` calcula a **sensação térmica**: a 32 °C com 70 % de umidade o corpo sente uns 40 °C, porque o suor evapora pouco.
- A saída no formato `Temperatura_C:28.0 Umidade_%:65` é entendida pelo **Plotter Serial** (Ferramentas → Plotter Serial), que desenha o gráfico sozinho.

Roteiro de teste — marque cada item

1. Ao ligar a USB, o LED do módulo acende.
2. Em 2 s aparecem as primeiras leituras e o contador de falhas fica em 0.
3. A temperatura está entre 20 e 35 °C e a umidade entre 30 e 90 % (valores de uma sala).
4. Segure o sensor entre os dedos: em 30 s a temperatura sobe 1–3 °C.
5. Sopre de leve sobre a grade: a umidade sobe vários pontos e volta em um minuto.
6. Tire o fio de dados: aparecem FALHAS. Recoloque: as leituras voltam sozinhas.

Teste no ESP32 com o celular

No ESP32 o módulo é alimentado em **3V3**: assim o pull-up do módulo puxa o fio de dados para 3,3 V, o nível certo para o GPIO. O DHT11 funciona de 3,3 a 5,5 V.

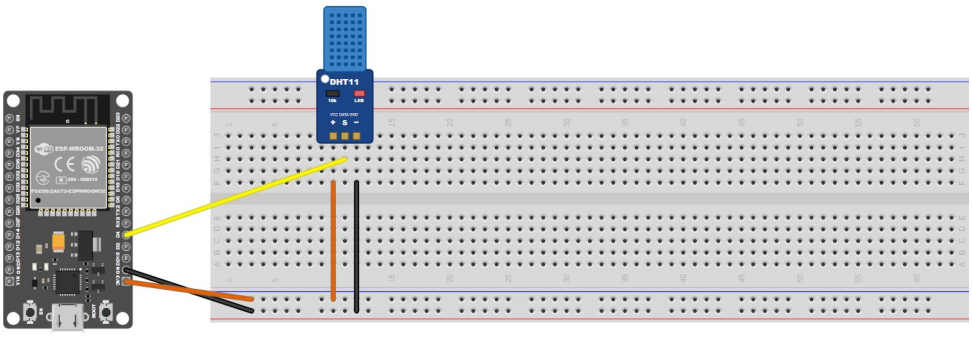


Figura 2 — Parte 1 — módulo DHT11 no ESP32: + no 3V3, dados no GPIO 4, − no GND.

O sketch do ESP32 cria a rede Wi-Fi **Ar-DHT11** (senha `12345678`). Conecte o celular a ela (desligue os dados móveis) e abra **http://192.168.4.1**: a página mostra o valor principal, os outros valores em cartões, um gráfico das últimas leituras e um aviso de **ALERTA**. Os arquivos `monitor_wifi.h` e `pagina_web.h` precisam estar **na mesma pasta** do sketch (a IDE os abre como abas).

01b_dht11_esp32_web.ino

```
/*
  01b - Modulo DHT11 (3 pinos) no ESP32, com monitor Wi-Fi no celular
  Placa: ESP32 Dev Module (DevKit V1, 30 pinos)    Biblioteca: DHT sensor library (Adafruit)
  Arquivos extras na mesma pasta: monitor_wifi.h, pagina_web.h

  Ligacoes:  + -> 3V3    DATA (meio) -> GPIO 4    - -> GND
  (o modulo funciona de 3,3 a 5,5 V; em 3V3 o sinal de dados ja sai no nivel certo do ESP32)
```

```
  Celular: conecte na rede "Ar-DHT11" (senha 12345678) e abra http://192.168.4.1
*/
#include <DHT.h>
#include "monitor_wifi.h"

const int PINO_DHT = 4;
const float UMIDADE_ALERTA = 70;      // % - acima disso, risco de mofo
const float TEMPERATURA_ALERTA = 30;  // C

DHT dht(PINO_DHT, DHT11);
unsigned long ultimaLeitura = 0, leituras = 0, falhas = 0;

void setup() {
  Serial.begin(115200);
  dht.begin();
  monitorIniciar("Ar-DHT11");
}

void loop() {
  monitorAtender();                                  // atende o celular sem parar
  if (millis() - ultimaLeitura < 2000) return;       // DHT11: 1 leitura a cada 2 s
  ultimaLeitura = millis();

  float h = dht.readHumidity(), t = dht.readTemperature();
  leituras++;
  if (isnan(h) || isnan(t)) {
    falhas++;
    Serial.printf("FALHA %lu de %lu - confira o fio DATA no GPIO 4 e a ordem + / -\n", falhas,
leituras);
    monitorPublicar(jsonMonitor("DHT11", item("Falhas", falhas, ""), true, "falha na leitura"));
    return;
  }
  float sensacao = dht.computeHeatIndex(t, h, false);
  bool alerta = h > UMIDADE_ALERTA || t > TEMPERATURA_ALERTA;
  Serial.printf("Temperatura_C:%.1f Umidade_%%:%.0f Sensacao_C:%.1f Falhas:%lu\n", t, h,
sensacao, falhas);
  monitorPublicar(jsonMonitor("DHT11 - modulo 3 pinos",
    item("Temperatura", t, "C") + "," + item("Umidade", h, "%") + "," + item("Sensacao",
sensacao, "C") + "," + item("Falhas", falhas, ""),
    alerta, h > UMIDADE_ALERTA ? "umidade alta: ventile" : "temperatura alta"));
}
```

- No ESP32 não usamos `delay(2000)`: o `loop()` precisa atender o celular o tempo todo. Por isso a leitura acontece quando `millis()` mostra que passaram 2 s.
- `monitorPublicar(jsonMonitor(...))` monta o JSON que a página lê: uma lista de itens `{n, v, u}` (nome, valor, unidade).

Parte 2 — DHT22 / AM2302 (3 fios)

O que muda em relação ao DHT11

O **DHT22** (vendido também como **AM2302** na versão com cabo) troca o elemento resistivo por um **sensor capacitivo**: um **polímero** entre duas placas absorve água, e como a água tem constante dielétrica alta (~80, contra ~3 do polímero seco), mais umidade significa **mais capacitância**. Isso dá leituras mais exatas e estáveis. O código é o mesmo: só troca `DHT11` por `DHT22`. A versão com fios já tem o **resistor de pull-up dentro da caixa**.

Característica	DHT11	DHT22 / AM2302
Faixa de temperatura	0 a 50 °C	−40 a 80 °C
Exatidão da temperatura	±2 °C	±0,5 °C
Faixa de umidade	20 a 90 %	0 a 100 %
Exatidão da umidade	±5 %	±2 a 5 %
Resolução	1 °C / 1 %	0,1 °C / 0,1 %
Intervalo mínimo entre leituras	1 s	2 s
Preço relativo	1×	3 a 4×

Ligação pelas cores dos fios

Fio	Função	Uno	ESP32
Vermelho	VCC (3,3–6 V)	5V	3V3
Amarelo	Dados	pino 2	GPIO 4
Preto	GND	GND	GND

As cores podem enganar

Quase todos os AM2302 seguem vermelho/amarelo/preto, mas há lotes com fios de outras cores. Na dúvida, procure a etiqueta na caixa do sensor ou meça: com o sensor ligado, o fio de dados fica em nível alto (perto do VCC) graças ao pull-up interno.

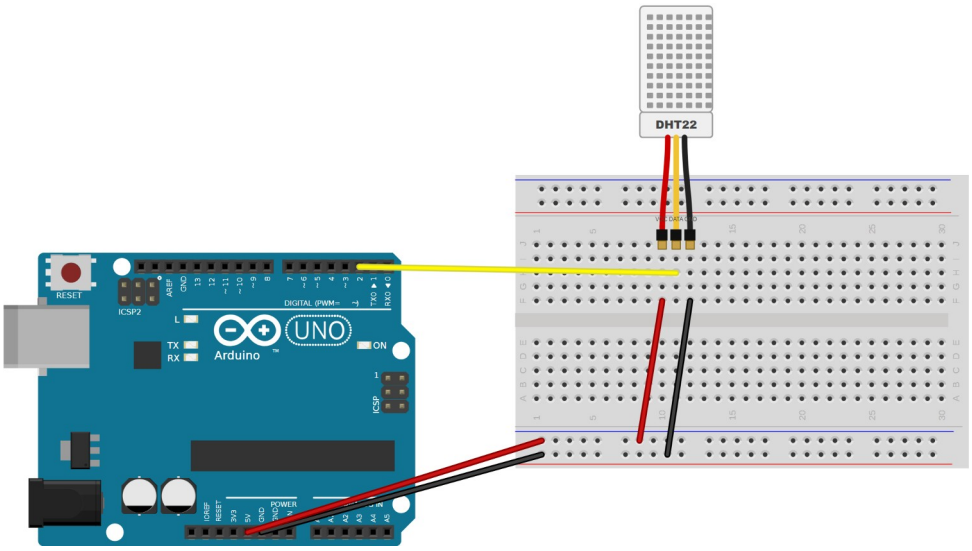


Figura 3 — Parte 2 — DHT22 com cabo no Uno: vermelho no 5 V, amarelo no pino 2, preto no GND.

02_dht22_uno.ino

```
/*
  02 - Teste do DHT22 / AM2302 com 3 fios (temperatura e umidade do ar)
  Placa: Arduino Uno/Nano     Bibliotecas: "DHT sensor library" (Adafruit) + "Adafruit Unified
Sensor"

  O seu sensor: caixinha branca com cabo de 3 fios.
     VERMELHO -> 5V        AMARELO (dados) -> pino 2        PRETO -> GND
  A versao com cabo (AM2302) ja tem o resistor de pull-up dentro. Se der muita "falha",
  coloque um resistor de 10 k entre o amarelo e o 5V.

  DHT22 x DHT11: faixa -40 a 80 C (+-0,5 C) e 0-100 % (+-2 a 5 %), resolucao 0,1.
  Le no maximo 1 vez a cada 2 s. Monitor Serial a 115200 baud (ou Plotter Serial).
*/
#include <DHT.h>

const int PINO_DHT = 2;
const unsigned long INTERVALO_MS = 2500;   // o DHT22 pede pelo menos 2 s entre leituras

DHT dht(PINO_DHT, DHT22);
unsigned long leituras = 0, falhas = 0;

// ponto de orvalho (aproximacao de Magnus): abaixo dessa temperatura a agua do ar condensa
float pontoDeOrvalho(float t, float ur) {
  const float a = 17.27, b = 237.7;
  float alfa = (a * t) / (b + t) + log(ur / 100.0);
  return (b * alfa) / (a - alfa);
}

void setup() {
  Serial.begin(115200);
  dht.begin();
  Serial.println(F("# Teste do DHT22 (3 fios) - aguarde 2,5 s pela primeira leitura"));
}

void loop() {
  delay(INTERVALO_MS);
  float umidade = dht.readHumidity();
  float temperatura = dht.readTemperature();
  leituras++;

  if (isnan(umidade) || isnan(temperatura)) {
    falhas++;
    Serial.print(F("# FALHA ("));
    Serial.print(falhas); Serial.print(F(" de ")); Serial.print(leituras);
    Serial.println(F(") - amarelo no pino 2? vermelho no 5V? se persistir, 10 k entre amarelo e
5V"));
    return;
  }
```

```
  Serial.print(F("Temperatura_C:")); Serial.print(temperatura, 1);
  Serial.print(F(" Umidade_%:"));    Serial.print(umidade, 1);
  Serial.print(F(" Orvalho_C:"));    Serial.print(pontoDeOrvalho(temperatura, umidade), 1);
  Serial.print(F(" Falhas:"));       Serial.println(falhas);
}
```

- `pontoDeOrvalho()` usa a **fórmula de Magnus**: é a temperatura em que o ar ficaria saturado e a água condensaria. Parede mais fria que o ponto de orvalho "sua" e cria mofo.
- O intervalo é de 2,5 s: ler o DHT22 mais rápido que 2 s faz a biblioteca devolver a leitura antiga ou NaN.

Roteiro de teste

1. Leituras sem falhas, com uma casa decimal.
2. Coloque o DHT11 e o DHT22 lado a lado por 10 min: anote a diferença de temperatura e de umidade. Ela cabe nas exatidões da tabela?
3. Bafo sobre a grade: o DHT22 reage mais rápido ou mais devagar que o DHT11?
4. Se aparecerem falhas frequentes com cabo comprido (> 1 m), coloque um resistor de 10 kΩ entre o amarelo e o vermelho.

Teste no ESP32

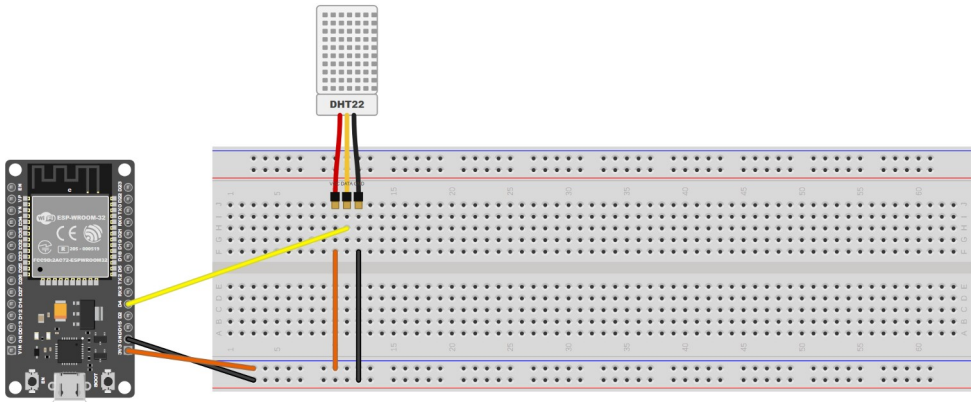


Figura 4 — Parte 2 — DHT22 no ESP32: vermelho no 3V3, amarelo no GPIO 4, preto no GND.

O sketch do ESP32 cria a rede Wi-Fi **Ar-DHT22** (senha `12345678`). Conecte o celular a ela (desligue os dados móveis) e abra **http://192.168.4.1**: a página mostra o valor principal, os outros valores em cartões, um gráfico das últimas leituras e um aviso de **ALERTA**. Os arquivos `monitor_wifi.h` e `pagina_web.h` precisam estar **na mesma pasta** do sketch (a IDE os abre como abas).

02b_dht22_esp32_web.ino

```
/*
  02b - DHT22 / AM2302 (3 fios) no ESP32, com monitor Wi-Fi no celular
  Placa: ESP32 Dev Module (DevKit V1, 30 pinos)    Biblioteca: DHT sensor library (Adafruit)
  Arquivos extras na mesma pasta: monitor_wifi.h, pagina_web.h

  Ligacoes:  VERMELHO -> 3V3    AMARELO -> GPIO 4    PRETO -> GND
  (o DHT22 funciona de 3,3 a 6 V; o pull-up interno do AM2302 fica no 3V3, nivel seguro para o
ESP32)

  Celular: rede "Ar-DHT22" (senha 12345678) -> http://192.168.4.1
*/
#include <DHT.h>
```

```
#include "monitor_wifi.h"

const int PINO_DHT = 4;
const float UMIDADE_ALERTA = 70, TEMPERATURA_ALERTA = 30;

DHT dht(PINO_DHT, DHT22);
unsigned long ultimaLeitura = 0, leituras = 0, falhas = 0;

float pontoDeOrvalho(float t, float ur) {
  const float a = 17.27, b = 237.7;
  float alfa = (a * t) / (b + t) + log(ur / 100.0);
  return (b * alfa) / (a - alfa);
}

void setup() {
  Serial.begin(115200);
  dht.begin();
  monitorIniciar("Ar-DHT22");
}

void loop() {
  monitorAtender();
  if (millis() - ultimaLeitura < 2500) return;       // DHT22: no minimo 2 s entre leituras
  ultimaLeitura = millis();

  float h = dht.readHumidity(), t = dht.readTemperature();
  leituras++;
  if (isnan(h) || isnan(t)) {
    falhas++;
    Serial.printf("FALHA %lu de %lu - amarelo no GPIO 4? vermelho no 3V3?\n", falhas, leituras);
    monitorPublicar(jsonMonitor("DHT22", item("Falhas", falhas, ""), true, "falha na leitura"));
    return;
  }
  float orvalho = pontoDeOrvalho(t, h);
  bool alerta = h > UMIDADE_ALERTA || t > TEMPERATURA_ALERTA;
  Serial.printf("Temperatura_C:%.1f Umidade_%%:%.1f Orvalho_C:%.1f Falhas:%lu\n", t, h, orvalho,
falhas);
  monitorPublicar(jsonMonitor("DHT22 - 3 fios",
    item("Temperatura", t, "C") + "," + item("Umidade", h, "%") + "," + item("Pto. orvalho",
orvalho, "C") + "," + item("Falhas", falhas, ""),
    alerta, h > UMIDADE_ALERTA ? "umidade alta: ventile" : "temperatura alta"));
}
```

Parte 3 — MQ-2: gás inflamável e fumaça

O que é e para que serve

O **MQ-2** detecta **GLP (gás de cozinha), propano, metano, hidrogênio, álcool e fumaça**. Serve para alarmes de vazamento de gás e de fumaça. Ele **não identifica** qual gás está presente nem mede concentração com exatidão: diz que **há algo diferente do ar limpo** e quanto a mais.

A química: óxido de estanho aquecido

Dentro da tela metálica há um tubinho de cerâmica coberto de **dióxido de estanho (SnO_2)** e uma resistência de **aquecimento** que o mantém a ~300 °C. No ar limpo, o **oxigênio** gruda na superfície do SnO_2 e "sequestra" elétrons: o material conduz mal e a resistência do sensor (**Rs**) é alta. Um gás **redutor** (GLP, H_2, álcool) reage com esse oxigênio, devolve os elétrons e a **resistência cai**. Por isso: **mais gás → Rs menor → tensão no AO maior**.

Do AO até a razão Rs/R0

O módulo tem um **resistor de carga RL** em série com o sensor, formando um divisor de tensão. O pino **AO** mostra a tensão sobre o RL. Lendo AO, calculamos o Rs:

```
Rs = RL × (Vc − Vao) / Vao          Vc = 5 V,  RL = resistor do módulo ("102" = 1 kΩ)
```

O Rs muda de um sensor para outro e com o tempo; por isso comparamos com o **R0**, o Rs que **o seu sensor** mede em **ar limpo** depois de aquecido. A **razão Rs/R0** fica perto de **1** no ar limpo e **cai** quando aparece gás. É exatamente essa razão que o projeto **ar** da Rede de Telemetria do Espaço CRIA publica, com a unidade `Rs/R0`.

E os "ppm" da folha de dados?

O fabricante dá curvas de Rs/R0 × concentração em escala log-log, mas o R0 da folha de dados é medido com um gás de referência, não em ar limpo. Convertemos (no ar limpo, Rs/R0 da folha = 9,83 no MQ-2) e mostramos uma **estimativa em ppm só como ordem de grandeza**. Para alarme, use a razão.

Pré-aquecimento: um sensor **novo** precisa de **24 a 48 h** ligado na primeira vez; no dia a dia, espere **pelo menos 3 min** (o ideal são 10 min) antes de confiar na leitura.

Montagem e teste no Uno

Pino do módulo	Uno	Observação
VCC	5V	aquecedor: ~150 mA
GND	GND	
DO	pino 3 (opcional)	0 quando passa do limiar do trimpot azul
AO	A0	tensão analógica

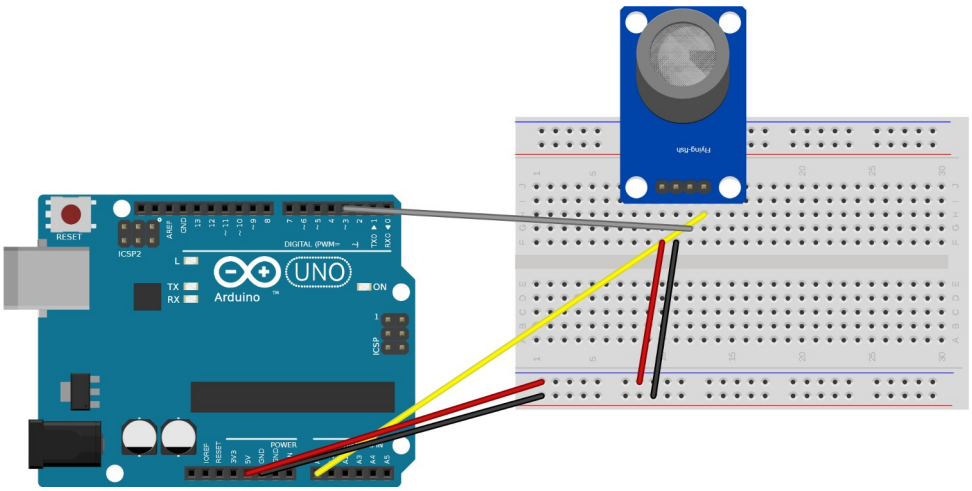


Figura 5 — Parte 3 — MQ-2 no Uno: VCC no 5 V, GND, DO no pino 3 e AO no A0.

03_mq2_uno.ino

```
/*
  03 - Teste do modulo MQ-2 (gas inflamavel: GLP, propano, H2, fumaca)
  Placa: Arduino Uno/Nano     Bibliotecas: nenhuma (fazemos a conta nos mesmos moldes da
telemetria-cria)

  Modulo de 4 pinos:  VCC -> 5V   GND -> GND   AO -> A0   DO -> pino 3 (opcional)
  O aquecedor puxa ~150 mA: pela USB funciona para UM sensor; para varios, use fonte 5 V externa.

  Como funciona: o elemento de SnO2 aquecido tem resistencia Rs. Gas redutor -> Rs CAI -> a
tensao
  em AO SOBE. O modulo tem um resistor de carga RL (leia na placa: "102" = 1 k, "103" = 10 k):
        Rs = RL * (Vc - Vao) / Vao
  R0 = Rs medido por NOS em ar limpo, depois do aquecimento. A razao Rs/R0 fica perto de 1 no ar
  limpo e cai com gas - e exatamente o que o projeto "ar" da telemetria-cria publica (unidade
Rs/R0).
  A estimativa em ppm usa a curva da folha de dados e serve so como ordem de grandeza.

  Passos do teste (Monitor Serial 115200 baud):
   1. Ligue e espere o PRE-AQUECIMENTO (contagem regressiva). Sensor novo: deixe 24-48 h ligado
antes.
   2. Se R0_KOHM = 0, o sketch mede o R0 sozinho por 30 s - FACA ISSO EM AR LIMPO (janela
aberta).
      Anote o valor mostrado e escreva em R0_KOHM para nao precisar recalibrar a cada vez.
   3. Teste: alcool 70% num algodao a 10 cm (NAO use isqueiro de botao: ele acende ao apertar).
Rs/R0 deve cair.
*/

const int PINO_AO = A0;
const int PINO_DO = 3;
const float VC = 5.0;               // tensao do modulo (meca com o multimetro e corrija)
const float RL_KOHM = 1.0;          // resistor de carga do modulo
float R0_KOHM = 0.0;                // 0 = calibrar ao ligar; depois escreva aqui o valor medido
const unsigned long PREAQUECIMENTO_S = 600;   // 10 min: o R0 so vale com o sensor estabilizado
const float FATOR_AR_LIMPO = 9.83;   // Rs/R0 da folha de dados no ar limpo
const float CURVA_A = 574.25, CURVA_B = -2.222;   // ppm = A * (Rs/R0_folha)^B  (GLP)
const float LIMIAR_RAZAO = 0.5;     // Rs/R0 abaixo disso: "gas detectado" (ajuste na sua sala)

// Le AO 20 vezes e devolve a tensao media (V); o ADC do Uno tem 10 bits (0-1023)
float tensaoAO() {
  long soma = 0;
  for (int i = 0; i < 20; i++) { soma += analogRead(PINO_AO); delay(2); }
  return (soma / 20.0) * VC / 1023.0;
}

// Rs em kOhm; devolve -1 se a tensao estiver fora da faixa (sensor desligado ou saturado)
float resistenciaSensor(float v) {
  if (v < 0.02 || v > VC - 0.02) return -1;
  return RL_KOHM * (VC - v) / v;
}

void setup() {
```

```
  Serial.begin(115200);
  pinMode(PINO_DO, INPUT);
  Serial.println(F("# Teste do MQ-2 - pre-aquecendo (o sensor esquenta: nao encoste)"));
  for (unsigned long s = PREAQUECIMENTO_S; s > 0; s--) {
    if (s % 15 == 0 || s <= 5) { Serial.print(F("# faltam ")); Serial.print(s); Serial.print(F("
s   AO = ")); Serial.print(tensaoAO(), 2); Serial.println(F(" V")); }
    delay(1000);
  }
  if (R0_KOHM <= 0) {
    Serial.println(F("# Medindo R0 por 30 s - mantenha o sensor em AR LIMPO..."));
    float soma = 0; int n = 0;
    for (int i = 0; i < 30; i++) {
      float rs = resistenciaSensor(tensaoAO());
      if (rs > 0) { soma += rs; n++; }
      delay(1000);
    }
    if (n == 0) { Serial.println(F("# ERRO: AO fora da faixa. Confira VCC/GND/AO e reinicie."));
while (true) {} }
    R0_KOHM = soma / n;
    Serial.print(F("# R0 = ")); Serial.print(R0_KOHM, 2);
    Serial.println(F(" kOhm  -> escreva esse valor em R0_KOHM no codigo"));
  }
  Serial.println(F("# AO_V, Rs_kOhm, Rs/R0, GLP_ppm_estimado, DO (0 = passou do limiar do
trimpot)"));
}

void loop() {
  float v = tensaoAO();
  float rs = resistenciaSensor(v);
  if (rs < 0) {
    Serial.print(F("# AO = ")); Serial.print(v, 2); Serial.println(F(" V fora da faixa: sensor
desligado ou saturado"));
    delay(1000);
    return;
  }
  float razao = rs / R0_KOHM;
  float ppm = CURVA_A * pow(razao * FATOR_AR_LIMPO, CURVA_B);
  int digital = digitalRead(PINO_DO);

  Serial.print(F("AO_V:"));     Serial.print(v, 3);
  Serial.print(F(" Rs_kOhm:")); Serial.print(rs, 2);
  Serial.print(F(" Rs/R0:"));   Serial.print(razao, 3);
  Serial.print(F(" GLP_ppm:"));   Serial.print(ppm, 0);
  Serial.print(F(" DO:"));      Serial.print(digital);
  if (razao < LIMIAR_RAZAO) Serial.print(F("  <<< GAS DETECTADO"));
  Serial.println();
  delay(1000);
}
```

Roteiro de teste

1. Leia o resistor RL do seu módulo (código de 3 algarismos perto do AO: `102` = 1 kΩ, `103` = 10 kΩ) e acerte `RL_KOHM`.
2. Ligue **perto de uma janela aberta** e espere a contagem do pré-aquecimento (10 min — aproveite para ler a química acima). A tensão de AO cai nos primeiros minutos e depois estabiliza.
3. O sketch mede o R0 por 30 s e o mostra. **Anote o R0** e escreva em `R0_KOHM`: você vai usá-lo no projeto final.
4. Rs/R0 deve ficar entre 0,9 e 1,1. Aproxime o algodão com álcool a 10 cm (ou peça ao professor a demonstração com o gás do isqueiro de pedra, sem chama): a razão despenca e aparece GAS DETECTADO.
5. Gire o trimpot azul até o LED do DO acender com o gás e apagar no ar limpo: esse é um alarme que funciona **sem microcontrolador**.

Teste no ESP32: o divisor de tensão

Com muito gás, o AO passa de 4 V — direto no ESP32, o pino queimaria. O **divisor 1 kΩ / 2 kΩ** entrega ao pino 2/3 da tensão: 4,5 V × 2/3 = **3,0 V**, que não danifica o pino (máximo 3,6 V). Mas o ADC só mede bem até 2,45 V: acima de AO ≈ 3,7 V o sketch avisa “saturou” (e trata como gás muito alto). O sketch desfaz a conta (multiplica por 3/2) e leva em conta que o divisor fica **em paralelo** com o RL do módulo. O **DO não é ligado** no ESP32: ele sai a 5 V.

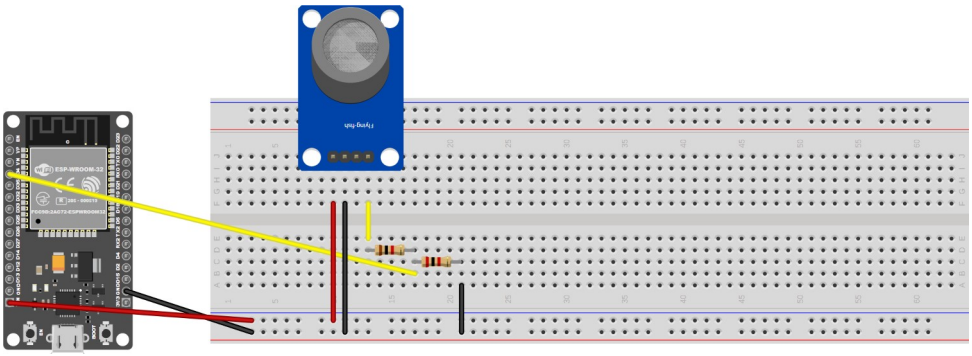


Figura 6 — Parte 3 — MQ-2 no ESP32: VCC no VIN (5 V), AO → 1 kΩ → GPIO 34 → 2 kΩ → GND (divisor montado na metade de baixo da protoboard).

O sketch do ESP32 cria a rede Wi-Fi **Ar-MQ2** (senha `12345678`). Conecte o celular a ela (desligue os dados móveis) e abra **http://192.168.4.1**: a página mostra o valor principal, os outros valores em cartões, um gráfico das últimas leituras e um aviso de **ALERTA**. Os arquivos `monitor_wifi.h` e `pagina_web.h` precisam estar **na mesma pasta** do sketch (a IDE os abre como abas).

03b_mq2_esp32_web.ino

```
/*
  03b - Modulo MQ-2 no ESP32, com monitor Wi-Fi no celular
  Placa: ESP32 Dev Module (DevKit V1, 30 pinos)    Bibliotecas: nenhuma extra
  Arquivos extras na mesma pasta: monitor_wifi.h, pagina_web.h

  Ligacoes:
    VCC -> VIN (5 V da USB)   GND -> GND
    AO  -> resistor 1 k -> GPIO 34 -> resistor 2 k -> GND      (divisor: o pino recebe 2/3 de AO)
    DO  -> NAO LIGAR no ESP32: a saida digital do modulo sobe a 5 V e queimaria o GPIO.
  O aquecedor precisa de 5 V; o AO pode chegar perto de 5 V, por isso o divisor e obrigatorio.

  A conta e a mesma do projeto "ar" da telemetria-cria (firmware/no-ar):
    tensao no pino -> tensao em AO -> Rs (levando em conta que o divisor fica em paralelo com o
RL)
    R0 = Rs em ar limpo, medido por este sketch apos o aquecimento -> publica Rs/R0.

  Celular: rede "Ar-MQ2" (senha 12345678) -> http://192.168.4.1
*/
#include "monitor_wifi.h"

const int PINO_AO = 34;                 // ADC1 (o ADC2 nao funciona com o Wi-Fi ligado)
const float VC = 5.0;                   // tensao do modulo (meca no VIN)
const float RL_KOHM = 1.0;              // resistor de carga do modulo ("102" = 1 k)
const float DIV_SUP_KOHM = 1.0, DIV_INF_KOHM = 2.0;   // divisor AO -> pino -> GND
float R0_KOHM = 0.0;                    // 0 = calibrar ao ligar; depois fixe o valor aqui
const unsigned long PREAQUECIMENTO_MS = 600000UL;    // 10 min: o R0 so vale com o sensor
estabilizado
const float FATOR_AR_LIMPO = 9.83;
const float CURVA_A = 574.25, CURVA_B = -2.222;   // GLP (folha de dados) - so ordem de grandeza
const float LIMIAR_RAZAO = 0.5;
// faixa em que o ADC do ESP32 (atenuacao 11 dB) mede com confianca
const uint32_t MENOR_MV = 150, MAIOR_MV = 2450;

enum Estado { AQUECENDO, CALIBRANDO, MEDINDO } estado = AQUECENDO;
unsigned long ultima = 0;
float somaR0 = 0; int nR0 = 0;

// tensao no pino em mV (media de 16 leituras: o ADC do ESP32 e ruidoso)
uint32_t milivoltsNoPino() {
```

```
  uint32_t soma = 0;
  for (int i = 0; i < 16; i++) soma += analogReadMilliVolts(PINO_AO);
  return soma / 16;
}

// Rs em kOhm a partir da tensao no pino; NAN se fora da faixa confiavel do ADC
float resistenciaSensor(uint32_t mv, float &vAO) {
  vAO = mv / 1000.0 * (DIV_SUP_KOHM + DIV_INF_KOHM) / DIV_INF_KOHM;
  if (mv < MENOR_MV || mv > MAIOR_MV || vAO >= VC) return NAN;
  float divisor = DIV_SUP_KOHM + DIV_INF_KOHM;
  float carga = RL_KOHM * divisor / (RL_KOHM + divisor);   // RL em paralelo com o divisor
  return carga * (VC - vAO) / vAO;
}

void setup() {
  Serial.begin(115200);
  analogSetPinAttenuation(PINO_AO, ADC_11db);
  monitorIniciar("Ar-MQ2");
  if (R0_KOHM > 0) Serial.printf("R0 fixo no codigo: %.2f kOhm\n", R0_KOHM);
}

void loop() {
  monitorAtender();
  if (millis() - ultima < 1000) return;
  ultima = millis();

  uint32_t mv = milivoltsNoPino();
  float vAO, rs = resistenciaSensor(mv, vAO);

  if (estado == AQUECENDO) {
    unsigned long falta = millis() < PREAQUECIMENTO_MS ? (PREAQUECIMENTO_MS - millis()) / 1000 :
0;
    Serial.printf("aquecendo: faltam %lu s  (pino %lu mV, AO %.2f V)\n", falta, (unsigned
long)mv, vAO);
    monitorPublicar(jsonMonitor("MQ-2 - pre-aquecimento", item("Faltam", falta, "s") + "," +
item("AO", vAO, "V"), false, "aguarde o aquecimento"));
    if (falta == 0) estado = (R0_KOHM > 0) ? MEDINDO : CALIBRANDO;
    return;
  }
  if (estado == CALIBRANDO) {
    if (!isnan(rs)) { somaR0 += rs; nR0++; }
    else if (mv < MENOR_MV) {   // ar limpo dando menos de 150 mV no pino: a calibracao nunca
terminaria
      Serial.printf("pino %lu mV: tensao baixa demais para o ADC. Meca o AO; com RL de 1 k,
troque o RL do modulo por 10 k (e RL_KOHM).\n", (unsigned long)mv);
      monitorPublicar(jsonMonitor("MQ-2 - AO baixo demais", item("Pino", mv, "mV"), true, "troque
o RL do modulo por 10 k"));
      return;
    }
    Serial.printf("calibrando R0 em ar limpo: %d/30  Rs = %.2f kOhm\n", nR0, rs);
    monitorPublicar(jsonMonitor("MQ-2 - calibrando R0 (ar limpo!)", item("Amostras", nR0, "/30")
+ "," + item("Rs", rs, "kOhm"), false, "mantenha em ar limpo"));
    if (nR0 >= 30) {
      R0_KOHM = somaR0 / nR0;
      Serial.printf(">>> R0 = %.2f kOhm  (escreva em R0_KOHM no codigo)\n", R0_KOHM);
      estado = MEDINDO;
    }
    return;
  }

  if (isnan(rs)) {
    bool saturado = mv > MAIOR_MV;
    Serial.printf("pino %lu mV fora da faixa do ADC: %s\n", (unsigned long)mv, saturado ? "gas
MUITO alto (saturou)" : "sensor desligado? fio do AO? RL?");
    monitorPublicar(jsonMonitor("MQ-2", item("Pino", mv, "mV"), saturado, saturado ? "GAS
DETECTADO (saturou)" : "sem leitura"));
    return;
  }
  float razao = rs / R0_KOHM;
  float ppm = CURVA_A * pow(razao * FATOR_AR_LIMPO, CURVA_B);
  bool alerta = razao < LIMIAR_RAZAO;
  Serial.printf("AO_V:%.3f Rs_kOhm:%.2f Rs/R0:%.3f GLP_ppm:%.0f%s\n", vAO, rs, razao, ppm, alerta
? "  <<< GAS DETECTADO" : "");
  monitorPublicar(jsonMonitor("MQ-2 - gas inflamavel",
    item("Rs/R0", razao, "") + "," + item("Rs", rs, "kOhm") + "," + item("AO", vAO, "V") + "," +
item("R0", R0_KOHM, "kOhm") + "," + item("GLP estim.", ppm, "ppm"),
```

```
    alerta, "GAS DETECTADO"));
}
```

- `analogReadMilliVolts()` já devolve a tensão em mV usando a calibração de fábrica do ADC do ESP32 — mais exato que `analogRead()`.
- O ADC do ESP32 só é confiável entre **150 e 2450 mV**. Fora disso o sketch avisa em vez de mostrar um número errado.
- O GPIO 34 é do **ADC1**. Os pinos do ADC2 param de funcionar quando o Wi-Fi está ligado.

Parte 4 — MQ-135: “ar viciado”

O que é e o que ele NÃO é

O **MQ-135** usa o mesmo SnO_2 aquecido, com outra dopagem: reage mais a **amônia (NH_3), álcool, benzeno, fumaça, óxidos de nitrogênio** e um pouco ao CO_2. Ele é bom para perceber **ar viciado** (sala cheia, cheiro de produto de limpeza, fumaça), mas **não mede CO_2**: qualquer um desses gases muda a leitura. No ar limpo, Rs/R0 da folha de dados = **3,6**.

> **Experimento da Parte 5**
> O sketch mostra um “CO_2 estimado” pela curva do fabricante (+400 ppm de base). Na Parte 5 você compara esse número com o **MH-Z19C**, que mede CO_2 de verdade por infravermelho. Adiantando: um vidro de álcool perto do MQ-135 faz o “CO_2” dele disparar, e o MH-Z19C nem percebe.

A ligação e as contas são **iguais às do MQ-2** (mesmo módulo de 4 pinos): VCC no 5 V, GND, AO no A0 (Uno) ou no divisor para o GPIO 34 (ESP32).

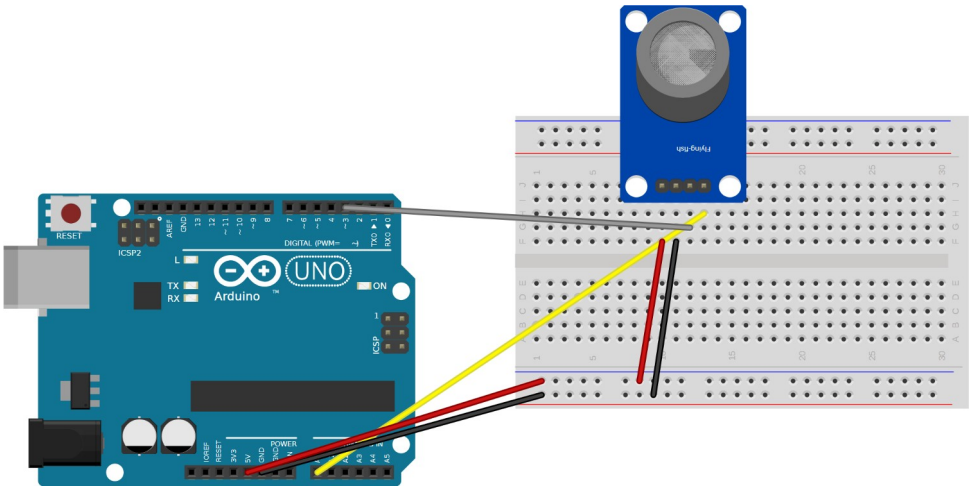


Figura 7 — Parte 4 — MQ-135 no Uno (mesma ligação do MQ-2).

04_mq135_uno.ino

```
/*
  04 - Teste do modulo MQ-135 (qualidade do ar: NH3, alcool, benzeno, fumaca, CO2 estimado)
  Placa: Arduino Uno/Nano     Bibliotecas: nenhuma (fazemos a conta nos mesmos moldes da
telemetria-cria)

  Modulo de 4 pinos:  VCC -> 5V   GND -> GND   AO -> A0   DO -> pino 3 (opcional)
  O aquecedor puxa ~150 mA: pela USB funciona para UM sensor; para varios, use fonte 5 V externa.

  Como funciona: o elemento de SnO2 aquecido tem resistencia Rs. Gas redutor -> Rs CAI -> a
tensao
  em AO SOBE. O modulo tem um resistor de carga RL (leia na placa: "102" = 1 k, "103" = 10 k):
        Rs = RL * (Vc - Vao) / Vao
  R0 = Rs medido por NOS em ar limpo, depois do aquecimento. A razao Rs/R0 fica perto de 1 no ar
  limpo e cai com gas - e exatamente o que o projeto "ar" da telemetria-cria publica (unidade
Rs/R0).
  A estimativa em ppm usa a curva da folha de dados e serve so como ordem de grandeza.
  ATENCAO: o MQ-135 NAO mede CO2 de verdade - ele reage a varios gases ao mesmo tempo. O "CO2"
  dele e um chute pela curva (+400 ppm de base). Na aula 5 comparamos com o MH-Z19C (NDIR), que
mede.

  Passos do teste (Monitor Serial 115200 baud):
   1. Ligue e espere o PRE-AQUECIMENTO (contagem regressiva). Sensor novo: deixe 24-48 h ligado
antes.
   2. Se R0_KOHM = 0, o sketch mede o R0 sozinho por 30 s - FACA ISSO EM AR LIMPO (janela
```

```
aberta).
      Anote o valor mostrado e escreva em R0_KOHM para nao precisar recalibrar a cada vez.
   3. Teste: bafo (CO2 + umidade), alcool num algodao ou vinagre a 10 cm. Rs/R0 deve cair.
*/

const int PINO_AO = A0;
const int PINO_DO = 3;
const float VC = 5.0;               // tensao do modulo (meca com o multimetro e corrija)
const float RL_KOHM = 1.0;          // resistor de carga do modulo
float R0_KOHM = 0.0;                // 0 = calibrar ao ligar; depois escreva aqui o valor medido
const unsigned long PREAQUECIMENTO_S = 600;   // 10 min: o R0 so vale com o sensor estabilizado
const float FATOR_AR_LIMPO = 3.6;   // Rs/R0 da folha de dados no ar limpo
const float CURVA_A = 110.47, CURVA_B = -2.862;   // ppm = A * (Rs/R0_folha)^B  (CO2)
const float LIMIAR_RAZAO = 0.7;     // Rs/R0 abaixo disso: "ar alterado" (ajuste na sua sala)

// Le AO 20 vezes e devolve a tensao media (V); o ADC do Uno tem 10 bits (0-1023)
float tensaoAO() {
  long soma = 0;
  for (int i = 0; i < 20; i++) { soma += analogRead(PINO_AO); delay(2); }
  return (soma / 20.0) * VC / 1023.0;
}

// Rs em kOhm; devolve -1 se a tensao estiver fora da faixa (sensor desligado ou saturado)
float resistenciaSensor(float v) {
  if (v < 0.02 || v > VC - 0.02) return -1;
  return RL_KOHM * (VC - v) / v;
}

void setup() {
  Serial.begin(115200);
  pinMode(PINO_DO, INPUT);
  Serial.println(F("# Teste do MQ-135 - pre-aquecendo (o sensor esquenta: nao encoste)"));
  for (unsigned long s = PREAQUECIMENTO_S; s > 0; s--) {
    if (s % 15 == 0 || s <= 5) { Serial.print(F("# faltam ")); Serial.print(s); Serial.print(F("
s   AO = ")); Serial.print(tensaoAO(), 2); Serial.println(F(" V")); }
    delay(1000);
  }
  if (R0_KOHM <= 0) {
    Serial.println(F("# Medindo R0 por 30 s - mantenha o sensor em AR LIMPO..."));
    float soma = 0; int n = 0;
    for (int i = 0; i < 30; i++) {
      float rs = resistenciaSensor(tensaoAO());
      if (rs > 0) { soma += rs; n++; }
      delay(1000);
    }
    if (n == 0) { Serial.println(F("# ERRO: AO fora da faixa. Confira VCC/GND/AO e reinicie."));
while (true) {} }
    R0_KOHM = soma / n;
    Serial.print(F("# R0 = ")); Serial.print(R0_KOHM, 2);
    Serial.println(F(" kOhm  -> escreva esse valor em R0_KOHM no codigo"));
  }
  Serial.println(F("# AO_V, Rs_kOhm, Rs/R0, CO2_ppm_estimado, DO (0 = passou do limiar do
trimpot)"));
}

void loop() {
  float v = tensaoAO();
  float rs = resistenciaSensor(v);
  if (rs < 0) {
    Serial.print(F("# AO = ")); Serial.print(v, 2); Serial.println(F(" V fora da faixa: sensor
desligado ou saturado"));
    delay(1000);
    return;
  }
  float razao = rs / R0_KOHM;
  float ppm = CURVA_A * pow(razao * FATOR_AR_LIMPO, CURVA_B) + 400;
  int digital = digitalRead(PINO_DO);

  Serial.print(F("AO_V:"));     Serial.print(v, 3);
  Serial.print(F(" Rs_kOhm:")); Serial.print(rs, 2);
  Serial.print(F(" Rs/R0:"));   Serial.print(razao, 3);
  Serial.print(F(" CO2_ppm:"));   Serial.print(ppm, 0);
  Serial.print(F(" DO:"));      Serial.print(digital);
  if (razao < LIMIAR_RAZAO) Serial.print(F("  <<< AR ALTERADO"));
  Serial.println();
  delay(1000);
```

```
}
```

Roteiro de teste

1. Pré-aquecimento e R0 em ar limpo, como no MQ-2. **Anote o R0 do MQ-135.**
2. Bafo a 10 cm por 5 s: Rs/R0 cai um pouco (umidade e CO_2 do ar expirado).
3. Algodão com álcool a 10 cm: a razão cai muito e aparece AR ALTERADO.
4. Afaste e cronometre quanto tempo leva para voltar a 0,9: o sensor tem **memória** (precisa "respirar" ar limpo).

Teste no ESP32

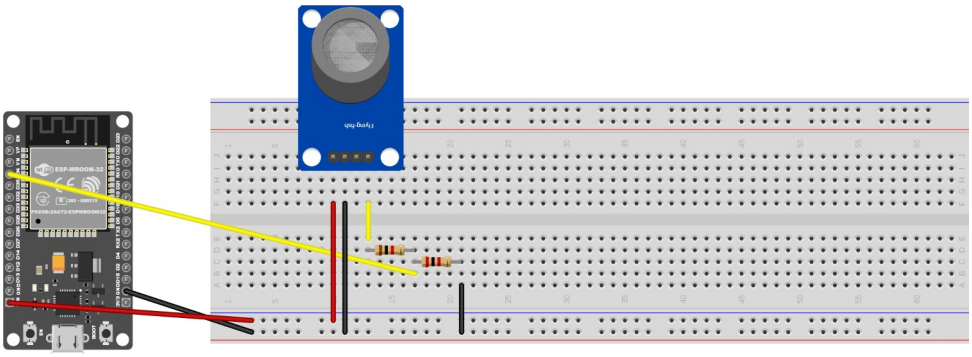


Figura 8 — Parte 4 — MQ-135 no ESP32 com divisor 1 kΩ / 2 kΩ no GPIO 34.

O sketch do ESP32 cria a rede Wi-Fi **Ar-MQ135** (senha `12345678`). Conecte o celular a ela (desligue os dados móveis) e abra **http://192.168.4.1**: a página mostra o valor principal, os outros valores em cartões, um gráfico das últimas leituras e um aviso de **ALERTA**. Os arquivos `monitor_wifi.h` e `pagina_web.h` precisam estar **na mesma pasta** do sketch (a IDE os abre como abas).

04b_mq135_esp32_web.ino

```
/*
  04b - Modulo MQ-135 no ESP32, com monitor Wi-Fi no celular
  Placa: ESP32 Dev Module (DevKit V1, 30 pinos)    Bibliotecas: nenhuma extra
  Arquivos extras na mesma pasta: monitor_wifi.h, pagina_web.h

  Ligacoes:
    VCC -> VIN (5 V da USB)   GND -> GND
    AO  -> resistor 1 k -> GPIO 34 -> resistor 2 k -> GND      (divisor: o pino recebe 2/3 de AO)
    DO  -> NAO LIGAR no ESP32: a saida digital do modulo sobe a 5 V e queimaria o GPIO.
  O aquecedor precisa de 5 V; o AO pode chegar perto de 5 V, por isso o divisor e obrigatorio.

  A conta e a mesma do projeto "ar" da telemetria-cria (firmware/no-ar):
    tensao no pino -> tensao em AO -> Rs (levando em conta que o divisor fica em paralelo com o
RL)
    R0 = Rs em ar limpo, medido por este sketch apos o aquecimento -> publica Rs/R0.

  Celular: rede "Ar-MQ135" (senha 12345678) -> http://192.168.4.1
*/
#include "monitor_wifi.h"

const int PINO_AO = 34;                 // ADC1 (o ADC2 nao funciona com o Wi-Fi ligado)
const float VC = 5.0;                   // tensao do modulo (meca no VIN)
const float RL_KOHM = 1.0;              // resistor de carga do modulo ("102" = 1 k)
const float DIV_SUP_KOHM = 1.0, DIV_INF_KOHM = 2.0;   // divisor AO -> pino -> GND
float R0_KOHM = 0.0;                    // 0 = calibrar ao ligar; depois fixe o valor aqui
const unsigned long PREAQUECIMENTO_MS = 600000UL;    // 10 min: o R0 so vale com o sensor
estabilizado
const float FATOR_AR_LIMPO = 3.6;
```

```
const float CURVA_A = 110.47, CURVA_B = -2.862;   // CO2 (folha de dados) - so ordem de grandeza
const float LIMIAR_RAZAO = 0.7;
// faixa em que o ADC do ESP32 (atenuacao 11 dB) mede com confianca
const uint32_t MENOR_MV = 150, MAIOR_MV = 2450;

enum Estado { AQUECENDO, CALIBRANDO, MEDINDO } estado = AQUECENDO;
unsigned long ultima = 0;
float somaR0 = 0; int nR0 = 0;

// tensao no pino em mV (media de 16 leituras: o ADC do ESP32 e ruidoso)
uint32_t milivoltsNoPino() {
  uint32_t soma = 0;
  for (int i = 0; i < 16; i++) soma += analogReadMilliVolts(PINO_AO);
  return soma / 16;
}

// Rs em kOhm a partir da tensao no pino; NAN se fora da faixa confiavel do ADC
float resistenciaSensor(uint32_t mv, float &vAO) {
  vAO = mv / 1000.0 * (DIV_SUP_KOHM + DIV_INF_KOHM) / DIV_INF_KOHM;
  if (mv < MENOR_MV || mv > MAIOR_MV || vAO >= VC) return NAN;
  float divisor = DIV_SUP_KOHM + DIV_INF_KOHM;
  float carga = RL_KOHM * divisor / (RL_KOHM + divisor);   // RL em paralelo com o divisor
  return carga * (VC - vAO) / vAO;
}

void setup() {
  Serial.begin(115200);
  analogSetPinAttenuation(PINO_AO, ADC_11db);
  monitorIniciar("Ar-MQ135");
  if (R0_KOHM > 0) Serial.printf("R0 fixo no codigo: %.2f kOhm\n", R0_KOHM);
}

void loop() {
  monitorAtender();
  if (millis() - ultima < 1000) return;
  ultima = millis();

  uint32_t mv = milivoltsNoPino();
  float vAO, rs = resistenciaSensor(mv, vAO);

  if (estado == AQUECENDO) {
    unsigned long falta = millis() < PREAQUECIMENTO_MS ? (PREAQUECIMENTO_MS - millis()) / 1000 :
0;
    Serial.printf("aquecendo: faltam %lu s  (pino %lu mV, AO %.2f V)\n", falta, (unsigned
long)mv, vAO);
    monitorPublicar(jsonMonitor("MQ-135 - pre-aquecimento", item("Faltam", falta, "s") + "," +
item("AO", vAO, "V"), false, "aguarde o aquecimento"));
    if (falta == 0) estado = (R0_KOHM > 0) ? MEDINDO : CALIBRANDO;
    return;
  }
  if (estado == CALIBRANDO) {
    if (!isnan(rs)) { somaR0 += rs; nR0++; }
    else if (mv < MENOR_MV) {   // ar limpo dando menos de 150 mV no pino: a calibracao nunca
terminaria
      Serial.printf("pino %lu mV: tensao baixa demais para o ADC. Meca o AO; com RL de 1 k,
troque o RL do modulo por 10 k (e RL_KOHM).\n", (unsigned long)mv);
      monitorPublicar(jsonMonitor("MQ-135 - AO baixo demais", item("Pino", mv, "mV"), true,
"troque o RL do modulo por 10 k"));
      return;
    }
    Serial.printf("calibrando R0 em ar limpo: %d/30  Rs = %.2f kOhm\n", nR0, rs);
    monitorPublicar(jsonMonitor("MQ-135 - calibrando R0 (ar limpo!)", item("Amostras", nR0,
"/30") + "," + item("Rs", rs, "kOhm"), false, "mantenha em ar limpo"));
    if (nR0 >= 30) {
      R0_KOHM = somaR0 / nR0;
      Serial.printf(">>> R0 = %.2f kOhm  (escreva em R0_KOHM no codigo)\n", R0_KOHM);
      estado = MEDINDO;
    }
    return;
  }

  if (isnan(rs)) {
    bool saturado = mv > MAIOR_MV;
    Serial.printf("pino %lu mV fora da faixa do ADC: %s\n", (unsigned long)mv, saturado ? "gas
MUITO alto (saturou)" : "sensor desligado? fio do AO? RL?");
```

```
    monitorPublicar(jsonMonitor("MQ-135", item("Pino", mv, "mV"), saturado, saturado ? "AR
ALTERADO: ventile (saturou)" : "sem leitura"));
    return;
  }
  float razao = rs / R0_KOHM;
  float ppm = CURVA_A * pow(razao * FATOR_AR_LIMPO, CURVA_B) + 400;
  bool alerta = razao < LIMIAR_RAZAO;
  Serial.printf("AO_V:%.3f Rs_kOhm:%.2f Rs/R0:%.3f CO2_ppm:%.0f%s\n", vAO, rs, razao, ppm, alerta
? "  <<< AR ALTERADO: ventile" : "");
  monitorPublicar(jsonMonitor("MQ-135 - qualidade do ar",
    item("Rs/R0", razao, "") + "," + item("Rs", rs, "kOhm") + "," + item("AO", vAO, "V") + "," +
item("R0", R0_KOHM, "kOhm") + "," + item("CO2 estim.", ppm, "ppm"),
    alerta, "AR ALTERADO: ventile"));
}
```

Parte 5 — MH-Z19C: CO_2 por infravermelho

O que é e para que serve

O **MH-Z19C** mede **CO_2 de 400 a 5000 ppm** (faixas de 400–2000, 400–5000 ou 400–10 000 ppm, configuráveis) com resolução de 1 ppm e exatidão de **±(50 ppm + 5 % da leitura)**, segundo o manual Winsen v1.2 (2021). Ele **aquece em 1 minuto** (o antigo MH-Z19B pedia 3) e responde em menos de 2 min (T90). É o tipo de sensor usado em monitores de ventilação de escolas: CO_2 alto numa sala fechada significa que o ar está sendo respirado de novo — e, junto com ele, aerossóis de quem está doente.

A física: NDIR — infravermelho não dispersivo

A molécula de CO_2 **absorve luz infravermelha** num comprimento de onda bem específico, **4,26 µm** (ela vibra nessa frequência). Dentro da caixa dourada há uma **lâmpada de infravermelho**, um tubo por onde o ar entra por difusão e um **detector com filtro** que só deixa passar 4,26 µm. Quanto mais CO_2 no tubo, **menos luz chega** ao detector — a relação segue a **lei de Beer-Lambert** (a intensidade cai exponencialmente com a concentração e o caminho). Como só o CO_2 absorve nesse comprimento, o sensor é **seletivo**: álcool, fumaça e umidade não enganam, ao contrário do MQ-135.

CO_2 (ppm)	Situação	O que fazer
~420	Ar livre	o zero do sensor é calibrado em 400 ppm
até 800	Sala bem ventilada	nada
800–1000	Moderado	ficar atento
1000–1500	Ar ruim (acima do valor máximo recomendável da ANVISA para ambientes climatizados)	abrir janelas
> 1500	Muito ruim: sonolência, dor de cabeça, menor atenção	ventilar já

Ligação e níveis de tensão

O MH-Z19C é **alimentado em 5,0 V ± 0,1 V** (média < 40 mA, **picos de 125 mA**), mas a sua comunicação serial (UART) e o PWM são de **3,3 V** (o manual diz "compatível com 5 V"). A versão com pinos tem **9 pinos em duas fileiras**, com os nomes impressos ao lado — **siga os nomes**, não a posição:

Fileira	Pino	Função	Uno	ESP32
1	HD	zero manual: 0 V por mais de 7 s calibra o zero em 400 ppm	**não ligar**	**não ligar**
1	SR	reservado de fábrica	—	—
1	Tx	UART: o sensor fala (3,3 V)	pino 10 direto	GPIO 16 (RX2) direto
1	Rx	UART: o sensor ouve (3,3 V)	pino 11 **pelo divisor** 1 kΩ/2 kΩ	GPIO 17 (TX2) direto
1	Vo	reservado de fábrica	—	—
2	PWM	saída PWM (ciclo 1004 ms; desafio 5)	—	—
2	AOT	saída analógica 0,4–2 V (não	—	A1 do ADS1115 (seção

Fileira	Pino	Função	Uno	ESP32
		documentada no manual; funciona)		seguinte)
2	GND	terra	GND	GND
2	Vin	5,0 V ± 0,1 V	5V	VIN (5 V)

Os 9 pinos do MH-Z19C

Vistos **por baixo**, com o sensor dourado para cima: fileira 1 = **HD, SR, Tx, Rx, Vo**; fileira 2 = **PWM, AOT, GND, Vin**. Nos esquemas deste guia os 9 pinos aparecem numa fileira só (para caber na protoboard): ligue pelos **nomes**.

No ESP32 ligado só pela USB, o VIN fica em ~4,6–4,8 V (há um diodo na placa), abaixo dos 4,5–5,5 V que o manual exige para a corrente de 150 mA — mais um motivo para a fonte de 5 V no projeto.

A versão com **conector** (terminal) tem 7 vias numeradas: 1 HD, 2 reservado, 3 GND, 4 Vin, 5 Rx, 6 Tx, 7 PWM.

O protocolo: 9 bytes e um checksum

A conversa é a 9600 baud. Para pedir a leitura, o Arduino manda 9 bytes; o sensor responde outros 9:

```
protocolo do MH-Z19C
pedido:   FF 01 86 00 00 00 00 00 79
resposta: FF 86 02 58 3C 00 00 00 E4
               ^^ ^^ CO2 = 0x02 * 256 + 0x58 = 512 + 88 = 600 ppm
                     ^^ temperatura interna = 0x3C - 40 = 20 C (grosseira)
checksum = 0xFF - (soma dos bytes 1 a 7) + 1 -> 0xFF - 0x1C + 1 = 0xE4 (a soma 0x11C perde o 1:
so cabe 1 byte)
```

No Uno, a única serial de hardware (pinos 0 e 1) está ocupada pela USB; por isso usamos a **SoftwareSerial** nos pinos 10 e 11. O ESP32 tem **três** seriais de hardware: usamos a **Serial2** nos GPIO 16/17 — os mesmos do nó de ar da telemetria-cria. O manual v1.2 do MH-Z19C documenta só dois comandos, `0x86` (ler) e `0x79` (ABC liga/desliga); o zero manual por comando (`0x87`) e a troca de faixa (`0x99`) vêm do manual do MH-Z19B e o C os aceita.

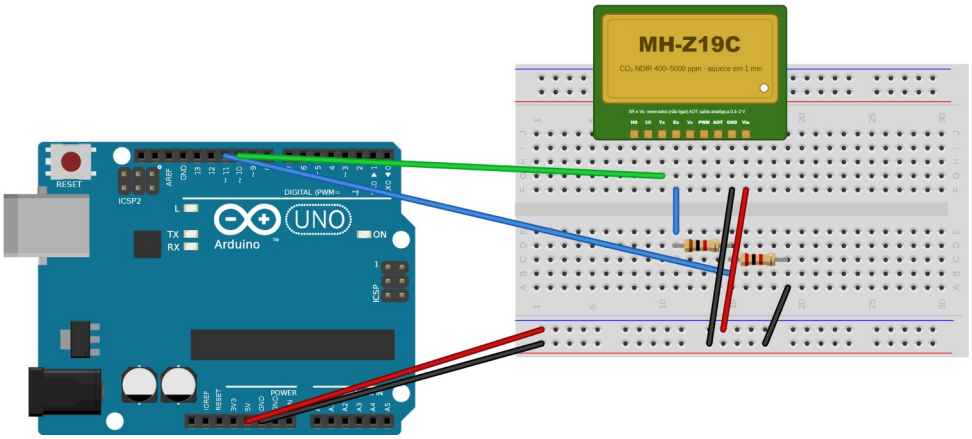


Figura 9 — Parte 5 — MH-Z19C no Uno: Vin no 5 V, GND; Tx do sensor no pino 10; pino 11 → 1 kΩ → Rx do sensor → 2 kΩ → GND (divisor na metade de baixo). HD, SR, Vo, PWM e AOT ficam livres.

```
05_mhz19c_uno.ino
/*
  05 - Teste do MH-Z19C: CO2 de verdade, por infravermelho (NDIR)
  Placa: Arduino Uno/Nano     Bibliotecas: SoftwareSerial (vem com o core AVR)
```

```
  MH-Z19C (manual Winsen v1.2): 9 pinos em DUAS fileiras - leia os NOMES na placa:
    fileira 1: HD  SR  Tx  Rx  Vo        fileira 2: PWM  AOT  GND  Vin
    SR e Vo sao reservados (nao ligar). HD = zero manual (0 V por 7 s: nao ligar!). AOT = saida
analogica 0,4-2 V.
  Ligacoes:
    Vin -> 5V (5,0 V +-0,1 V; pico 125 mA)        GND -> GND
    TX do sensor -> pino 10 do Uno (direto: 3,3 V passa do limiar de 3,0 V do Uno - funciona, com
margem pequena)
    RX do sensor <- pino 11 do Uno ATRAVES do divisor 1 k / 2 k:
         pino 11 -> 1 k -> RX do sensor -> 2 k -> GND      (5 V viram 3,3 V; a UART do sensor e
de 3,3 V)

  Protocolo: 9600 baud. Pedido de 9 bytes  FF 01 86 00 00 00 00 00 79
             Resposta de 9 bytes          FF 86 HH LL TT SS -- -- CS   (CO2 = HH*256 + LL)
             checksum = 0xFF - (soma dos bytes 1 a 7) + 1

  Comandos pelo Monitor Serial (115200 baud, "Nova linha"):
    z  calibrar o ZERO em 400 ppm  (so ao ar livre, sensor ligado ha 20-30 min ou mais!)
    a  liga a autocalibracao ABC   (o sensor assume que, 1 vez a cada 24 h, o ar chega a 400 ppm)
    d  desliga a ABC               (recomendado em sala que fica sempre fechada)
    2  faixa 400-2000 ppm   5  faixa 400-5000 ppm
  (o manual v1.2 do MH-Z19C documenta so 0x86 e 0x79; z (0x87) e 2/5 (0x99) sao do manual do MH-
Z19B e o C os aceita)
*/
#include <SoftwareSerial.h>

SoftwareSerial co2Serial(10, 11);       // RX (recebe do TX do sensor), TX (vai ao RX do sensor)
const unsigned long PREAQUECIMENTO_MS = 60000UL;    // 1 min (manual do MH-Z19C v1.2; o MH-Z19B
pedia 3 min)
const unsigned long INTERVALO_MS = 5000;
unsigned long ultima = 0, falhas = 0;
bool aquecido = false;

// monta o pacote de 9 bytes com o checksum e envia
void enviarComando(uint8_t cmd, uint8_t b3 = 0, uint8_t b6 = 0, uint8_t b7 = 0) {
  uint8_t p[9] = {0xFF, 0x01, cmd, b3, 0x00, 0x00, b6, b7, 0x00};
  uint8_t soma = 0;
  for (int i = 1; i < 8; i++) soma += p[i];
  p[8] = 0xFF - soma + 1;
  while (co2Serial.available()) co2Serial.read();    // descarta restos
  co2Serial.write(p, 9);
}

// pede a leitura; devolve o CO2 em ppm ou -1 (sem resposta / checksum errado)
int lerCO2(int &temperatura, uint8_t &status) {
  enviarComando(0x86);
  uint8_t r[9];
  if (co2Serial.readBytes(r, 9) != 9) return -1;
  if (r[0] != 0xFF || r[1] != 0x86) return -1;
  uint8_t soma = 0;
  for (int i = 1; i < 8; i++) soma += r[i];
  if ((uint8_t)(0xFF - soma + 1) != r[8]) return -1;
  temperatura = (int)r[4] - 40;       // temperatura interna (grosseira, nao use como termometro)
  status = r[5];
  return r[2] * 256 + r[3];
}

const char* qualidade(int ppm) {
  if (ppm <= 800) return "boa";
  if (ppm <= 1000) return "moderada";
  if (ppm <= 1500) return "ruim - ventile";
  return "muito ruim - abra janelas";
}

void setup() {
  Serial.begin(115200);
  co2Serial.begin(9600);
  co2Serial.setTimeout(300);
  Serial.println(F("# Teste do MH-Z19C. Comandos: z (zero 400 ppm), a/d (ABC liga/desliga), 2/5
(faixa)"));
}

void loop() {
  if (Serial.available()) {
```

```
    char c = Serial.read();
    if (c == 'z') { enviarComando(0x87); Serial.println(F("# ZERO calibrado em 400 ppm (so vale
se estava ao ar livre!)")); }
    if (c == 'a') { enviarComando(0x79, 0xA0); Serial.println(F("# ABC ligada")); }
    if (c == 'd') { enviarComando(0x79, 0x00); Serial.println(F("# ABC desligada")); }
    if (c == '2') { enviarComando(0x99, 0, 2000 >> 8, 2000 & 0xFF); Serial.println(F("# faixa 0-
2000 ppm")); }
    if (c == '5') { enviarComando(0x99, 0, 5000 >> 8, 5000 & 0xFF); Serial.println(F("# faixa 0-
5000 ppm")); }
  }
  if (millis() - ultima < INTERVALO_MS) return;
  ultima = millis();

  int temperatura; uint8_t status;
  int ppm = lerCO2(temperatura, status);
  if (ppm < 0) {
    falhas++;
    Serial.print(F("# sem resposta do sensor (")); Serial.print(falhas);
    Serial.println(F(") - TX do sensor no pino 10? RX no 11 pelo divisor? Vin em 5V?"));
    return;
  }
  if (millis() >= PREAQUECIMENTO_MS) aquecido = true;
  if (!aquecido) {
    Serial.print(F("# aquecendo (")); Serial.print((PREAQUECIMENTO_MS - millis()) / 1000);
    Serial.print(F(" s): leitura ainda nao vale -> ")); Serial.println(ppm);
    return;
  }
  Serial.print(F("CO2_ppm:")); Serial.print(ppm);
  Serial.print(F(" Temp_interna_C:")); Serial.print(temperatura);
  Serial.print(F("  ar ")); Serial.println(qualidade(ppm));
}
```

Calibração: zero, ABC e cuidados

- **Pré-aquecimento:** 1 min (manual v1.2). Antes disso o sensor costuma responder valores fixos (400, 410, 500) — o sketch avisa.
- **ABC (autocalibração):** vem ligada. O sensor guarda o menor valor de cada 24 h e assume que ele era 400 ppm. Ótimo para casas e salas que ficam vazias e arejadas à noite; **ruim** para uma sala que fica fechada sempre (ele "aprende" um zero errado). Comandos `a` e `d`.
- **Zero manual (comando `z`, ou HD no GND por 7 s):** só **ao ar livre**, com o sensor ligado há **20–30 min ou mais** (o manual pede ambiente estável a 400 ppm por mais de 30 min) e longe de pessoas (nosso bafo tem ~40 000 ppm de CO_2!). O manual recomenda recalibrar **a cada 6 meses**.
- Respostas sem sentido (sempre 5000, ou sempre o mesmo valor) podem indicar **sensor falsificado** — há cópias no mercado. Compare dois sensores lado a lado.

Roteiro de teste

1. Sem resposta? Troque Tx e Rx de lugar (é o erro mais comum), confira o divisor e se não ligou por engano em SR ou Vo.
2. Após 1 min, numa sala com janela aberta: 420–600 ppm.
3. Sopre uma vez de perto: o valor sobe a milhares de ppm em 30–60 s e volta devagar (o ar entra por **difusão**).
4. Feche a porta e a janela com a turma dentro e registre o CO_2 a cada 5 min. Abra tudo e veja a queda.
5. Coloque o MQ-135 ao lado e aproxime um algodão com álcool: compare o "CO_2 estimado" do MQ-135 com o MH-Z19C.

Teste no ESP32

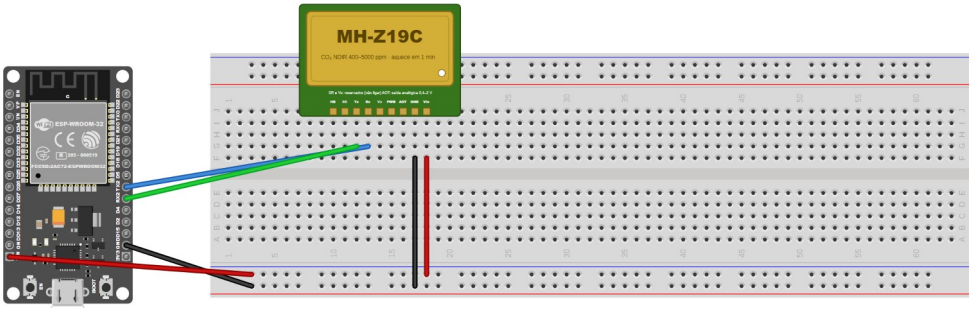


Figura 10 — Parte 5 — MH-Z19C no ESP32: Vin no VIN (5 V), GND; Tx → GPIO 16 (RX2), Rx → GPIO 17 (TX2), sem divisor.

O sketch do ESP32 cria a rede Wi-Fi **Ar-CO2** (senha `12345678`). Conecte o celular a ela (desligue os dados móveis) e abra **http://192.168.4.1**: a página mostra o valor principal, os outros valores em cartões, um gráfico das últimas leituras e um aviso de **ALERTA**. Os arquivos `monitor_wifi.h` e `pagina_web.h` precisam estar **na mesma pasta** do sketch (a IDE os abre como abas).

05b_mhz19c_esp32_web.ino

```
/*
  05b - MH-Z19C (CO2 por infravermelho) no ESP32, com monitor Wi-Fi no celular
  Placa: ESP32 Dev Module (DevKit V1, 30 pinos)    Bibliotecas: nenhuma extra (Serial2 vem no
core)
  Arquivos extras na mesma pasta: monitor_wifi.h, pagina_web.h

  MH-Z19C: 9 pinos em duas fileiras (HD SR Tx Rx Vo | PWM AOT GND Vin). SR e Vo reservados; HD
nao ligar.
  Ligacoes (as mesmas do no-ar da telemetria-cria):
    Vin -> VIN (5 V)   GND -> GND
    TX do sensor -> GPIO 16 (RX2)      RX do sensor -> GPIO 17 (TX2)
  A UART do sensor e de 3,3 V: liga DIRETO no ESP32, sem divisor.

  Comandos pelo Monitor Serial (115200): z = zero 400 ppm (ar livre, 20 min ligado), a/d = ABC
liga/desliga
  Celular: rede "Ar-CO2" (senha 12345678) -> http://192.168.4.1
*/
#include "monitor_wifi.h"

const int PINO_RX2 = 16, PINO_TX2 = 17;
const unsigned long PREAQUECIMENTO_MS = 60000UL;    // 1 min (MH-Z19C)
const int CO2_ALERTA = 1000;            // ppm: limite recomendado para ambiente interno (ANVISA
RE 9/2003)
unsigned long ultima = 0, falhas = 0;
int maximo = 0;
bool aquecido = false;

void enviarComando(uint8_t cmd, uint8_t b3 = 0, uint8_t b6 = 0, uint8_t b7 = 0) {
  uint8_t p[9] = {0xFF, 0x01, cmd, b3, 0x00, 0x00, b6, b7, 0x00};
  uint8_t soma = 0;
  for (int i = 1; i < 8; i++) soma += p[i];
  p[8] = 0xFF - soma + 1;
  while (Serial2.available()) Serial2.read();
  Serial2.write(p, 9);
}

int lerCO2(int &temperatura) {
  enviarComando(0x86);
  uint8_t r[9];
  if (Serial2.readBytes(r, 9) != 9) return -1;
  if (r[0] != 0xFF || r[1] != 0x86) return -1;
  uint8_t soma = 0;
  for (int i = 1; i < 8; i++) soma += r[i];
  if ((uint8_t)(0xFF - soma + 1) != r[8]) return -1;
  temperatura = (int)r[4] - 40;
  return r[2] * 256 + r[3];
}

const char* qualidade(int ppm) {
  return ppm <= 800 ? "boa" : ppm <= 1000 ? "moderada" : ppm <= 1500 ? "ruim - ventile" : "muito
ruim";
```

```
}

void setup() {
  Serial.begin(115200);
  Serial2.begin(9600, SERIAL_8N1, PINO_RX2, PINO_TX2);
  Serial2.setTimeout(150);   // curto: enquanto espera, a pagina do celular nao e atendida
  monitorIniciar("Ar-CO2");
}

void loop() {
  monitorAtender();
  if (Serial.available()) {
    char c = Serial.read();
    if (c == 'z') { enviarComando(0x87); Serial.println("ZERO calibrado em 400 ppm"); }
    if (c == 'a') { enviarComando(0x79, 0xA0); Serial.println("ABC ligada"); }
    if (c == 'd') { enviarComando(0x79, 0x00); Serial.println("ABC desligada"); }
  }
  if (millis() - ultima < 5000) return;
  ultima = millis();


  int temperatura;
  int ppm = lerCO2(temperatura);
  if (ppm < 0) {
    falhas++;
    Serial.printf("sem resposta do MH-Z19C (%lu) - TX->GPIO16? RX->GPIO17? Vin no VIN (5 V)?\n",
falhas);
    monitorPublicar(jsonMonitor("MH-Z19C", item("Falhas", falhas, ""), true, "sensor nao
responde"));
    return;
  }
  if (millis() >= PREAQUECIMENTO_MS) aquecido = true;
  if (!aquecido) {
    unsigned long falta = (PREAQUECIMENTO_MS - millis()) / 1000;
    Serial.printf("aquecendo (%lu s): %d ppm ainda nao vale\n", falta, ppm);
    monitorPublicar(jsonMonitor("MH-Z19C - aquecendo", item("Faltam", falta, "s"), false,
"aguarde 3 min"));
    return;
  }
  if (ppm > maximo) maximo = ppm;
  Serial.printf("CO2_ppm:%d Temp_interna_C:%d  ar %s\n", ppm, temperatura, qualidade(ppm));
  monitorPublicar(jsonMonitor("MH-Z19C - CO2 (NDIR)",
    item("CO2", ppm, "ppm") + "," + item("Maximo", maximo, "ppm") + "," + item("Temp. interna",
temperatura, "C"),
    ppm > CO2_ALERTA, String("ar ") + qualidade(ppm) + ": abra as janelas"));
}
```

Conversor A/D ADS1115: por que e como usar nesta aula

O problema: o ADC do ESP32

Dois sensores desta aula são **analógicos**: os MQ entregam uma tensão no AO. No Uno o ADC de 10 bits (degraus de 4,9 mV, razoavelmente linear) dá conta. No ESP32 o ADC interno de 12 bits é **não linear**, só é confiável entre **~0,15 e 2,45 V**, satura perto de 3,1 V e sofre com o **ruído do Wi-Fi** — por isso o sketch 03b avisa "fora da faixa" quando o AO cai a 0,2 V em ar limpo (RL de 1 kΩ), e por isso o R0 medido no ESP32 nunca bate com o do Uno. Para um alarme de gás isso basta; para publicar `Rs/R0` num painel que compara nós de salas diferentes, não.

A solução: um conversor A/D externo de 16 bits

O **ADS1115** (Texas Instruments) é um conversor analógico-digital **sigma-delta de 16 bits** (65 536 níveis, 1 bit de sinal) com **4 entradas** (ou 2 pares diferenciais), um **amplificador de ganho programável (PGA)** e interface **I2C**. Ele mede a tensão com exatidão e **manda o número pronto** pelo barramento — o ruído do fio não altera um número que viaja em bits. É o conversor que o nó **`horta/no1`** da Rede de Telemetria usa para pH, condutividade e pressão, e serve igualmente para o AO dos MQ e para a saída analógica **AOT** do MH-Z19C.

	ADC do Uno	ADC do ESP32	ADS1115
Bits / níveis	10 / 1024	12 / 4096	16 / 65 536
Degrau	4,9 mV	0,8 mV	0,125 mV (ganho 1) · 0,1875 mV (ganho 2/3)
Linearidade	boa	ruim (satura, não linear)	excelente
Faixa útil	0–5 V	0,15–2,45 V	até VDD + 0,3 V na entrada
Velocidade	~10 000/s	~100 000/s	8 a 860/s
Entradas	6	ADC1 (o ADC2 morre com Wi-Fi)	4 (ou 2 diferenciais)

Ganho (PGA)	Fundo de escala	1 nível	Quando usar
2/3	±6,144 V	0,1875 mV	sinal até 5 V com o ADS1115 **em 5 V** (nó `horta/no1`)
1	±4,096 V	0,125 mV	sinal até 3,3 V — nosso sketch `03c`
2 / 4 / 8 / 16	±2,048 … ±0,256 V	62 µV … 7,8 µV	sinais pequenos (o AOT do MH-Z19C, 0,4–2 V, cabe no ganho 2)

Dois limites que confundem todo mundo

O PGA **não aumenta a tensão que a entrada aguenta**: qualquer entrada deve ficar entre GND − 0,3 V e **VDD + 0,3 V**. Com o ADS1115 em 3,3 V, o AO dos MQ (até 5 V) **precisa do divisor** 1 kΩ/2 kΩ, mesmo que o ganho 2/3 "chegue" a 6,144 V.

Alimentado em **5 V** ele lê o AO direto — mas o I2C sobe para 5 V e o ESP32 não tolera: é preciso um **conversor de nível lógico** em SDA/SCL. É assim que o nó `horta/no1` está montado (ADS1115 em 5 V, ganho 2/3, A0 pH, A1

condutividade, A2 pressão, conversor de nível entre o ADS e os GPIO 21/22).

Como ligar (versão simples: ADS1115 em 3,3 V)

ADS1115	ESP32	
VDD	3V3	2,0–5,5 V; consome 150 µA
GND	GND	
SCL	GPIO 22	I2C do projeto (os mesmos pinos do DS3231 na telemetria)
SDA	GPIO 21	
ADDR	GND	endereço **0x48** (VDD = 0x49, SDA = 0x4A, SCL = 0x4B: até 4 módulos)
A0	AO do MQ-2 → 1 kΩ → A0 → 2 kΩ → GND	o sketch desfaz o divisor (× 1,5)
A1	AOT do MH-Z19C (opcional)	0,4–2 V: liga direto
ALRT	—	não usamos

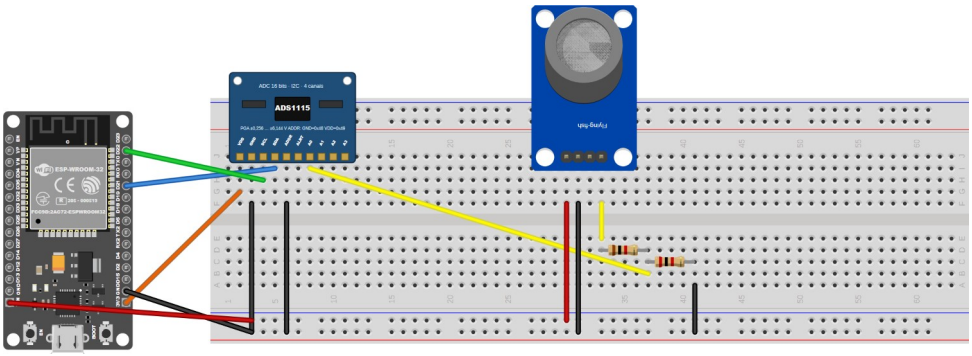


Figura 11 — MQ-2 lido pelo ADS1115 no ESP32: ADS1115 em 3,3 V (VDD, GND, SCL→22, SDA→21, ADDR→GND); AO → divisor 1 kΩ/2 kΩ → A0.

O sketch do ESP32 cria a rede Wi-Fi **Ar-MQ2-ADS** (senha `12345678`). Conecte o celular a ela (desligue os dados móveis) e abra **http://192.168.4.1**: a página mostra o valor principal, os outros valores em cartões, um gráfico das últimas leituras e um aviso de **ALERTA**. Os arquivos `monitor_wifi.h` e `pagina_web.h` precisam estar **na mesma pasta** do sketch (a IDE os abre como abas).

03c_mq2_esp32_ads1115_web.ino

```
/*
  03c - Modulo MQ-2 no ESP32 lido pelo ADS1115 (conversor A/D de 16 bits, I2C) + monitor Wi-Fi
  Placa: ESP32 Dev Module     Biblioteca: "Adafruit ADS1X15" (instala junto a Adafruit BusIO)
  Arquivos extras: monitor_wifi.h, pagina_web.h


  POR QUE: o ADC interno do ESP32 (12 bits) e nao linear, so e confiavel entre ~0,15 e 2,45 V e
sofre com o ruido do
  Wi-Fi. Com o AO do MQ perto de 0,2 V em ar limpo (RL de 1 k), o sketch 03b ja avisa "fora da
faixa". O ADS1115 tem
  16 bits, e linear, tem amplificador programavel (PGA) e manda o NUMERO pronto por I2C. E o
conversor que o no
  horta/no1 da Rede de Telemetria do Espaco CRIA usa para pH, condutividade e pressao.
```

```
  COMO (versao simples, ADS1115 em 3,3 V - o I2C ja fica no nivel do ESP32, sem conversor de
nivel):
    ADS1115  VDD -> 3V3   GND -> GND   SCL -> GPIO 22   SDA -> GPIO 21   ADDR -> GND (endereco
0x48)
    MQ-2     VCC -> VIN (5 V)   GND -> GND   AO -> 1 k -> A0 do ADS1115 -> 2 k -> GND   (divisor:
as entradas do
             ADS1115 nao podem passar de VDD + 0,3 V; o AO chega a 5 V)
    DO       nao ligar.
  Com ganho 1 (+-4,096 V) cada nivel vale 0,125 mV no pino = 0,19 mV no AO: 25x melhor que o ADC
do ESP32 na pratica.
  (No no da telemetria o ADS1115 e alimentado em 5 V, le o sinal direto com ganho 2/3 e um
conversor de nivel logico
   protege o I2C do ESP32.)  Para ler tambem o AOT (0,4-2 V) do MH-Z19C, ligue-o direto no A1:
cabe em 3,3 V.


  A conta de Rs e Rs/R0 e a mesma do 03b (o divisor fica em paralelo com o RL do modulo).
  Celular: rede "Ar-MQ2-ADS" (senha 12345678) -> http://192.168.4.1
*/
#include <Adafruit_ADS1X15.h>
#include "monitor_wifi.h"


const uint8_t CANAL_AO = 0;
const float VC = 5.0;                   // tensao do modulo (meca no VIN)
const float RL_KOHM = 1.0;              // resistor de carga do modulo ("102" = 1 k)
const float DIV_SUP_KOHM = 1.0, DIV_INF_KOHM = 2.0;
const float FATOR_DIVISOR = (DIV_SUP_KOHM + DIV_INF_KOHM) / DIV_INF_KOHM;   // 1,5
float R0_KOHM = 0.0;                    // 0 = medir ao ligar (apos o aquecimento); depois fixe o
valor aqui
const unsigned long PREAQUECIMENTO_MS = 600000UL;
const float LIMIAR_GAS = 0.5;           // Rs/R0 abaixo disso = gas
Adafruit_ADS1115 ads;
bool adsPresente = false;
unsigned long ultima = 0, inicioR0 = 0; float somaR0 = 0; int nR0 = 0;


float tensaoAO() {                      // volts no AO do modulo (desfaz o divisor); NAN se o
ADS1115 nao responder
  if (!adsPresente) return NAN;
  long soma = 0;
  for (int i = 0; i < 8; i++) {
    ads.startADCReading(MUX_BY_CHANNEL[CANAL_AO], false);
    unsigned long t0 = millis();
    while (!ads.conversionComplete()) { if (millis() - t0 > 25) return NAN; delay(1); }
    soma += ads.getLastConversionResults();
  }
  return ads.computeVolts((int16_t)(soma / 8)) * FATOR_DIVISOR;
}
float resistenciaRs(float vao) {        // carga real = RL em paralelo com o divisor (1 k + 2 k)
  float d = DIV_SUP_KOHM + DIV_INF_KOHM, carga = RL_KOHM * d / (RL_KOHM + d);
  if (vao < 0.01 || vao > VC - 0.01) return NAN;
  return carga * (VC - vao) / vao;
}

void setup() {
  Serial.begin(115200);
  Serial.setTimeout(50);
  Wire.begin(21, 22);
  adsPresente = ads.begin(0x48, &Wire);
  if (adsPresente) { ads.setGain(GAIN_ONE); ads.setDataRate(RATE_ADS1115_128SPS);
Serial.println("ADS1115 em 0x48: ok (ganho 1: 0,125 mV/nivel)"); }
  else Serial.println("ERRO: ADS1115 nao responde em 0x48 - confira SDA/SCL/VDD e o ADDR no
GND");
  monitorIniciar("Ar-MQ2-ADS");
}

void loop() {
  monitorAtender();
  if (millis() - ultima < 2000) return;
  ultima = millis();
  float vao = tensaoAO(), rs = resistenciaRs(vao);
  bool aquecido = millis() >= PREAQUECIMENTO_MS;
  if (isnan(vao)) { monitorPublicar(jsonMonitor("MQ-2 (ADS1115)", item("Falha", 1, ""), true,
"ADS1115 nao responde")); return; }
  if (!aquecido) {
    Serial.printf("aquecendo: faltam %lu s   AO=%.4f V  Rs=%.2f k\n", (PREAQUECIMENTO_MS -
millis()) / 1000, vao, rs);
    monitorPublicar(jsonMonitor("MQ-2 pelo ADS1115 - aquecendo", item("Faltam",
(PREAQUECIMENTO_MS - millis()) / 1000, "s") + "," + item("AO", vao, "V"), false));
```

```
    return;
  }
  if (R0_KOHM == 0 && !isnan(rs)) {     // 30 s de media em ar limpo logo apos o aquecimento
    if (inicioR0 == 0) inicioR0 = millis();
    somaR0 += rs; nR0++;
    if (millis() - inicioR0 >= 30000) { R0_KOHM = somaR0 / nR0; Serial.printf("R0 medido: %.2f k
(escreva em R0_KOHM)\n", R0_KOHM); }
  }
  float razao = (R0_KOHM > 0 && !isnan(rs)) ? rs / R0_KOHM : NAN;
  bool gas = !isnan(razao) && razao < LIMIAR_GAS;
  Serial.printf("AO:%.4f V  Rs:%.2f k  Rs/R0:%.3f%s\n", vao, rs, razao, gas ? "  <<< GAS
DETECTADO" : "");   // 4 casas: o ADS1115 enxerga 0,2 mV
  monitorPublicar(jsonMonitor("MQ-2 pelo ADS1115",
    item("Rs/R0", razao, "") + "," + item("AO", vao, "V") + "," + item("Rs", rs, "k") + "," +
item("R0", R0_KOHM, "k"),
    gas, "gas detectado"));
}
```

Entenda o código

- `ads.begin(0x48, &Wire)` procura o chip no barramento; se não responder, o sketch avisa e segue (sem travar o Wi-Fi).
- `setGain(GAIN_ONE)` (±4,096 V) e `128 SPS` (8 ms por conversão). `startADCReading` + `conversionComplete` com **prazo de 25 ms** é a leitura sem bloquear do firmware `horta-no1`: um fio solto no I2C não pode travar o nó.
- `computeVolts()` converte a contagem em volts pelo ganho; × 1,5 desfaz o divisor. A conta de Rs continua considerando o divisor **em paralelo** com o RL.
- Repare na saída `AO:0.2031 V`: quatro casas com sentido. No ADC interno a terceira já era ruído.

Experimento: ESP32 × ADS1115

1. Com o MQ-2 aquecido em ar limpo, grave `03b` e anote 20 leituras de AO; depois `03c` e anote outras 20. Compare o espalhamento (maior − menor).
2. Aproxime o celular baixando algo (Wi-Fi em uso) do ESP32: o ADC interno mexe? E o ADS1115?
3. Meça o R0 com os dois: qual se aproxima do R0 medido no Uno (sketch 03)?
4. Ligue o AOT do MH-Z19C no A1 (ganho 2) e compare a tensão com o ppm da serial em três situações (janela aberta, sala fechada, bafo): a relação é linear?

Parte 6 — Projeto: monitor de qualidade do ar

Agora juntamos os cinco sensores e um **LED RGB** que mostra a qualidade do ar à distância. O projeto tem duas versões: no **Uno**, os dados saem pela USB e um programa no computador (o **gateway**) os envia ao servidor; no **ESP32**, a placa envia sozinha pelo Wi-Fi. Nas duas, os dados aparecem no **painel da Rede de Telemetria do Espaço CRIA**, junto com os outros projetos da escola.

Cor do LED	Significado
Branco rápido ao ligar	teste do LED
Azul fixo	aquecendo o MH-Z19C (1 min)
Verde	CO_2 até 800 ppm: ar bom
Amarelo (vermelho + verde)	CO_2 de 800 a 1000 ppm: atenção
Vermelho fixo	CO_2 acima de 1000 ppm: abra a janela
Vermelho piscando	o MQ-2 detectou gás (tem prioridade sobre o resto)
Azul piscando	o MH-Z19C não respondeu
Roxo piscando (só ESP32)	sem conexão com o servidor: as leituras ficam guardadas na fila

A Rede de Telemetria do Espaço CRIA em uma página

A plataforma `telemetria-cria` é o servidor da escola: um **broker MQTT** (Mosquitto) recebe as mensagens, o **InfluxDB** guarda as séries de tempo e um **painel web** mostra tudo ao vivo e exporta CSV. Cada medida tem um endereço (tópico) e uma mensagem em JSON:

catálogo de tópicos (telemetria-cria/mqtt/catalogo-topicos.md)

```
cria/{projeto}/{no}/{grandeza}        ex.: cria/ar/no2/co2
{"valor": 612, "unidade": "ppm", "seq": 1834, "boot": 7, "t": 1793732797772}
```

Grandeza	Sensor	Unidade	Quando é publicada
temperatura	DHT22	C	a cada 10 s (leitura com falha não sai)
umidade	DHT22	%	a cada 10 s
co2	MH-Z19C	ppm	depois de 1 min de aquecimento
mq2	MQ-2	Rs/R0	depois de 10 min e só com o R0 medido
mq135	MQ-135	Rs/R0	idem

E o DHT11?

O catálogo não permite duas grandezas `temperatura` no mesmo nó. Publicamos o DHT22, que é mais exato, e o DHT11 aparece no Monitor Serial **ao lado** dele, com a diferença — ótimo para avaliar o DHT11. Se quiser o DHT11 no painel, publique-o por outro nó (outro número) no projeto `teste`.

6A — Monitor no Arduino Uno

Peça	Ligação no Uno
Trilhas da protoboard	+ = 5 V da **fonte 5 V/2 A**, − = GND. Ligue **só o GND** da fonte ao GND do Uno; **não** ligue o pino 5V do Uno nessa trilha (duas fontes em paralelo empurram corrente para a USB do PC). Sem fonte (só para testar), use o 5V do Uno e nada mais
DHT11 (módulo)	+ no +, DATA (meio) no **D2**, − no −
DHT22 (fios)	vermelho no +, amarelo no **D4**, preto no −
MQ-2	VCC no +, GND no −, AO no **A0**
MQ-135	VCC no +, GND no −, AO no **A1**
MH-Z19C	Vin no +, GND no −, TX no **D10**, RX ← divisor ← **D11**
LED RGB (cátodo comum)	R → 220 Ω → **D9**, G → 220 Ω → **D6**, B → 220 Ω → **D5**, cátodo no −

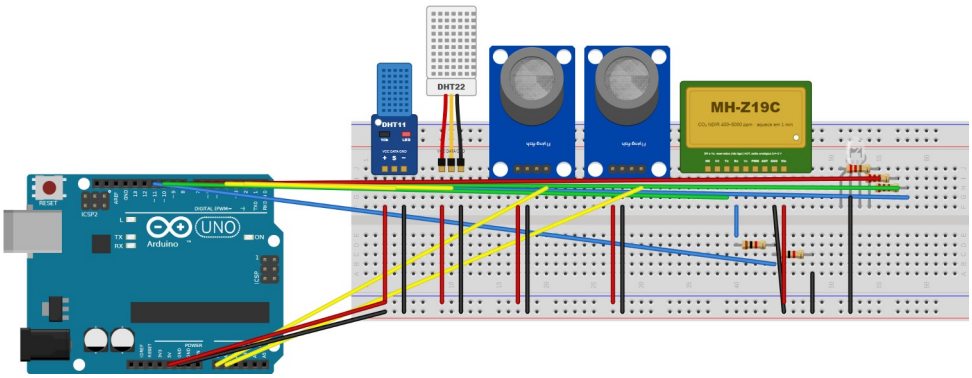


Figura 12 — Parte 6A — monitor completo no Uno: DHT11, DHT22, MQ-2, MQ-135, MH-Z19C (com o divisor no RX) e LED RGB.

1. Monte **um sensor por vez** e teste com o sketch individual antes de passar ao próximo.
2. Abra `06_monitor_ar_uno`, escreva os **R0** que você mediu nas Partes 3 e 4 (`R0_MQ2_KOHM`, `R0_MQ135_KOHM`) e confira os RL.
3. Com `SAIDA_GATEWAY 0`, grave e abra o Monitor Serial: uma linha a cada 10 s com todos os sensores lado a lado.
4. Espere 10 min com a janela aberta: tudo estabilizado, LED verde.

06_monitor_ar_uno.ino

```
/*
  06 - PROJETO: Monitor de qualidade do ar no Arduino Uno (todos os sensores juntos)
  Placa: Arduino Uno     Bibliotecas: "DHT sensor library" + "Adafruit Unified Sensor";
SoftwareSerial (core)


  Ligacoes (protoboard de 830 pontos; trilha + = 5 V, trilha - = GND):
    DHT11 (modulo 3 pinos)  + -> 5V   DATA (meio) -> D2   - -> GND
    DHT22 (3 fios)          vermelho -> 5V   amarelo -> D4   preto -> GND
    MQ-2                    VCC -> 5V   GND -> GND   AO -> A0
    MQ-135                  VCC -> 5V   GND -> GND   AO -> A1
    MH-Z19C                 Vin -> 5V   GND -> GND   TX -> D10   RX <- divisor (D11 -> 1 k -> RX
-> 2 k -> GND)
    LED RGB (catodo comum)  R -> 220 R -> D9   G -> 220 R -> D6   B -> 220 R -> D5   catodo ->
GND


  ALIMENTACAO: 2 aquecedores MQ (~150 mA cada) + MH-Z19C (picos de 125 mA) + Uno ~ 0,5 A. A USB
do PC
  fornece 0,5 A no limite: alimente os SENSORES por uma FONTE 5 V / 2 A nas trilhas da protoboard
e ligue
  so o GND da fonte ao GND do Uno. NAO ligue a fonte no pino 5V do Uno com a USB conectada
(empurra
  corrente para a porta do PC). Meca o 5V do Uno e escreva em VC (o ADC usa o 5V do Uno como
referencia).
```

```
  LED de status: AZUL = aquecendo | VERDE = CO2 ate 800 ppm | AMARELO = ate 1000 | VERMELHO =
acima de 1000
                 VERMELHO PISCANDO = MQ-2 detectou gas (prioridade) | AZUL PISCANDO = MH-Z19C sem
resposta

  SAIDA na USB (115200 baud):
    SAIDA_GATEWAY 0 -> tabela legivel para o Monitor Serial
    SAIDA_GATEWAY 1 -> uma linha JSON por leitura, no formato do gateway da telemetria-cria
                       {"no":"no3","grandeza":"co2","valor":612,"unidade":"ppm","seq":12}
                       Rode gateway/gateway.py (projeto no .env do gateway) e os dados aparecem
no painel.
  Grandezas publicadas (catalogo, projeto "ar"): temperatura e umidade (DHT22), co2 (MH-Z19C),
  mq2 e mq135 (Rs/R0, so depois do aquecimento e com R0 medido). O DHT11 aparece so na tabela,
  para comparar com o DHT22: o catalogo nao deixa duas "temperatura" no mesmo no.
*/
#include <DHT.h>
#include <SoftwareSerial.h>

// ------------------------------------------------------------------ ajustes
#define SAIDA_GATEWAY 0
const char* NO = "no3";                // no deste monitor no catalogo: escolha um livre
(painel)!

const int PINO_DHT11 = 2, PINO_DHT22 = 4;
const int PINO_MQ2 = A0, PINO_MQ135 = A1;
const int PINO_CO2_RX = 10, PINO_CO2_TX = 11;
const int LED_R = 9, LED_G = 6, LED_B = 5;

const float VC = 5.0;                  // tensao dos MQ (meca)
const float RL_MQ2_KOHM = 1.0, RL_MQ135_KOHM = 1.0;
// R0 de cada MQ em ar limpo, em kOhm: use o valor que os sketches 03 e 04 mediram.
// Enquanto valer 0, o Rs aparece na tabela e o MQ NAO e publicado.
const float R0_MQ2_KOHM = 0.0, R0_MQ135_KOHM = 0.0;
const float LIMIAR_GAS_MQ2 = 0.5;      // Rs/R0 do MQ-2 abaixo disso = gas

const unsigned long INTERVALO_MS = 10000;                // padrao do catalogo
const unsigned long PREAQUECIMENTO_CO2_MS = 1UL * 60 * 1000;   // MH-Z19C: 1 min
const unsigned long PREAQUECIMENTO_MQ_MS = 10UL * 60 * 1000;
const int FAIXA_CO2_PPM = 5000;
// ------------------------------------------------------------------------

DHT dht11(PINO_DHT11, DHT11);
DHT dht22(PINO_DHT22, DHT22);
SoftwareSerial co2Serial(PINO_CO2_RX, PINO_CO2_TX);

unsigned long seq = 0, ultimaLeitura = 0, ultimoPisca = 0;
bool jaLeu = false, piscaAceso = false;
bool co2Aquecido = false, mqAquecidos = false;   // ficam verdadeiras para sempre (sobrevivem a
volta do millis)
int ultimoCO2 = -1;
bool gasDetectado = false;

// ------------------------------------------------------------------ sensores
int lerCO2() {
  const uint8_t pedido[9] = {0xFF, 0x01, 0x86, 0x00, 0x00, 0x00, 0x00, 0x00, 0x79};
  while (co2Serial.available()) co2Serial.read();
  co2Serial.write(pedido, 9);
  uint8_t r[9];
  if (co2Serial.readBytes(r, 9) != 9 || r[0] != 0xFF || r[1] != 0x86) return -1;
  uint8_t soma = 0;
  for (int i = 1; i < 8; i++) soma += r[i];
  if ((uint8_t)(0xFF - soma + 1) != r[8]) return -1;
  return r[2] * 256 + r[3];
}

// Rs (kOhm) de um MQ ligado direto ao Uno; NAN se a tensao estiver fora da faixa
float resistenciaMq(int pino, float rl) {
  long soma = 0;
  for (int i = 0; i < 20; i++) soma += analogRead(pino);
  float v = (soma / 20.0) * VC / 1023.0;
  if (v < 0.02 || v > VC - 0.02) return NAN;
  return rl * (VC - v) / v;
}
```

```
// ------------------------------------------------------------------ saida
// Uma linha JSON por leitura (formato do gateway). Leitura com falha nao sai e nao recebe seq.
void publicar(const char* grandeza, float valor, uint8_t casas, const char* unidade) {
  if (isnan(valor) || isinf(valor)) return;
#if SAIDA_GATEWAY
  seq++;
  Serial.print(F("{\"no\":\""));       Serial.print(NO);
  Serial.print(F("\",\"grandeza\":\"")); Serial.print(grandeza);
  Serial.print(F("\",\"valor\":"));     Serial.print(valor, casas);
  Serial.print(F(",\"unidade\":\""));   Serial.print(unidade);
  Serial.print(F("\",\"seq\":"));       Serial.print(seq);
  Serial.println(F("}"));
#else
  (void)grandeza; (void)valor; (void)casas; (void)unidade;
#endif
}

void coluna(const __FlashStringHelper* nome, float v, uint8_t casas) {
#if !SAIDA_GATEWAY
  Serial.print(nome);
  if (isnan(v)) Serial.print(F("  --  ")); else Serial.print(v, casas);
  Serial.print(F("  "));
#else
  (void)nome; (void)v; (void)casas;
#endif
}

// ------------------------------------------------------------------ LED
void cor(bool r, bool g, bool b) { digitalWrite(LED_R, r); digitalWrite(LED_G, g);
digitalWrite(LED_B, b); }

void atualizarLed() {
  if (millis() - ultimoPisca < 300) return;
  ultimoPisca = millis();
  piscaAceso = !piscaAceso;
  if (gasDetectado)                        cor(piscaAceso, 0, 0);
  else if (!co2Aquecido)                   cor(0, 0, 1);
  else if (ultimoCO2 < 0)                  cor(0, 0, piscaAceso);
  else if (ultimoCO2 <= 800)               cor(0, 1, 0);
  else if (ultimoCO2 <= 1000)              cor(1, 1, 0);
  else                                     cor(1, 0, 0);
}

// ------------------------------------------------------------------ programa
void setup() {
  Serial.begin(115200);
  pinMode(LED_R, OUTPUT); pinMode(LED_G, OUTPUT); pinMode(LED_B, OUTPUT);
  cor(1, 1, 1); delay(300); cor(0, 0, 0);     // teste rapido do LED: branco
  dht11.begin();
  dht22.begin();
  co2Serial.begin(9600);
  co2Serial.setTimeout(300);
  // linha que nao comeca com "{": o gateway so mostra no terminal
  Serial.print(F("telemetria: monitor de ar ")); Serial.println(NO);
}

void loop() {
  atualizarLed();
  if (jaLeu && millis() - ultimaLeitura < INTERVALO_MS) return;
  jaLeu = true;
  ultimaLeitura = millis();

  float t11 = dht11.readTemperature(), h11 = dht11.readHumidity();
  float t22 = dht22.readTemperature(), h22 = dht22.readHumidity();
  if (millis() >= PREAQUECIMENTO_CO2_MS) co2Aquecido = true;
  if (millis() >= PREAQUECIMENTO_MQ_MS) mqAquecidos = true;
  int co2 = lerCO2();
  if (co2 <= 0 || co2 > FAIXA_CO2_PPM) co2 = -1;   // fora da faixa = falha
  ultimoCO2 = co2Aquecido ? co2 : -1;

  float rs2 = resistenciaMq(PINO_MQ2, RL_MQ2_KOHM);
  float rs135 = resistenciaMq(PINO_MQ135, RL_MQ135_KOHM);
  float razao2 = (R0_MQ2_KOHM > 0 && mqAquecidos) ? rs2 / R0_MQ2_KOHM : NAN;
  float razao135 = (R0_MQ135_KOHM > 0 && mqAquecidos) ? rs135 / R0_MQ135_KOHM : NAN;
```

```
  gasDetectado = !isnan(razao2) && razao2 < LIMIAR_GAS_MQ2;

  // --- telemetria (so com SAIDA_GATEWAY 1)
  publicar("temperatura", t22, 1, "C");
  publicar("umidade", h22, 1, "%");
  if (co2Aquecido && co2 > 0) publicar("co2", co2, 0, "ppm");
  publicar("mq2", razao2, 3, "Rs/R0");
  publicar("mq135", razao135, 3, "Rs/R0");

  // --- tabela (so com SAIDA_GATEWAY 0)
  coluna(F("DHT11 T:"), t11, 1);  coluna(F("U:"), h11, 0);
  coluna(F("| DHT22 T:"), t22, 1); coluna(F("U:"), h22, 1);
  coluna(F("| CO2:"), co2Aquecido && co2 > 0 ? co2 : NAN, 0);
  coluna(F("| MQ2 Rs:"), rs2, 2);  coluna(F("Rs/R0:"), razao2, 2);
  coluna(F("| MQ135 Rs:"), rs135, 2); coluna(F("Rs/R0:"), razao135, 2);
#if !SAIDA_GATEWAY
  if (!co2Aquecido) Serial.print(F("| CO2 aquecendo "));
  if (!mqAquecidos) Serial.print(F("| MQ aquecendo "));
  if (gasDetectado) Serial.print(F("| <<< GAS!"));
  Serial.println();
#endif
}
```

Enviando o Uno para o painel (gateway USB)

O Uno não tem rede. Com `#define SAIDA_GATEWAY 1`, ele escreve na USB **uma linha JSON por leitura**, e o programa `gateway/gateway.py` da telemetria-cria, no computador ao lado, acrescenta a hora e o projeto e envia ao servidor.

1. Troque para `#define SAIDA_GATEWAY 1`, escolha um **nó livre** em `NO` (veja no painel quais existem) e grave.
2. No computador, na pasta `telemetria-cria`, copie `.env.example` para `.env` e preencha `PORTA_SERIAL` (ex.: `/dev/ttyACM0`), `SERVIDOR_URL` (ex.: `http://192.168.0.10:3000`), `GATEWAY_PROJETO` (`ar`, ou `teste` nas oficinas) e `GATEWAY_SENHA` (peça ao professor).
3. **Feche o Monitor Serial** (a porta só aceita um programa) e rode `python3 gateway/gateway.py`.
4. O terminal mostra `placa: telemetria: monitor de ar no3` e, a cada 10 s, as leituras enviadas. Abra o painel: o nó aparece no projeto escolhido.

terminal

```
$ python3 gateway/gateway.py
14:02:10  gateway do projeto "ar" enviando para http://192.168.0.10:3000. Ctrl+C encerra.
14:02:11  placa: telemetria: monitor de ar no3
14:02:21  5 leituras enviadas (no3 boot 1); seq até 5.
```

6B — Monitor no ESP32, publicando direto no servidor

O ESP32 usa a biblioteca **CriaTelemetria**, a biblioteca-padrão da rede: ela conecta no Wi-Fi, no broker MQTT, acerta o relógio pelo servidor da escola, numera as leituras (`seq` e `boot`) e **guarda na memória flash** o que não puder ser enviado quando a rede cair. O nosso código só lê os sensores e chama `no.publicar("co2", co2, "ppm")`.

Peça	Ligação no ESP32
Trilha + (baixo)	**VIN (5 V)** — de preferência alimentado por fonte 5 V/2 A; trilha − = GND
DHT22 (fios)	vermelho no **3V3**, amarelo no **GPIO 4**, preto no −

Peça	Ligação no ESP32
DHT11 (módulo)	+ no **3V3**, DATA no **GPIO 13**, − no −
MQ-2	VCC no +, GND no −, AO → 1 kΩ → **GPIO 34** → 2 kΩ → −
MQ-135	VCC no +, GND no −, AO → 1 kΩ → **GPIO 35** → 2 kΩ → −
MH-Z19C	Vin no +, GND no −, TX → **GPIO 16**, RX → **GPIO 17**
LED RGB	R → 220 Ω → **GPIO 25**, G → 220 Ω → **GPIO 26**, B → 220 Ω → **GPIO 27**, cátodo no −

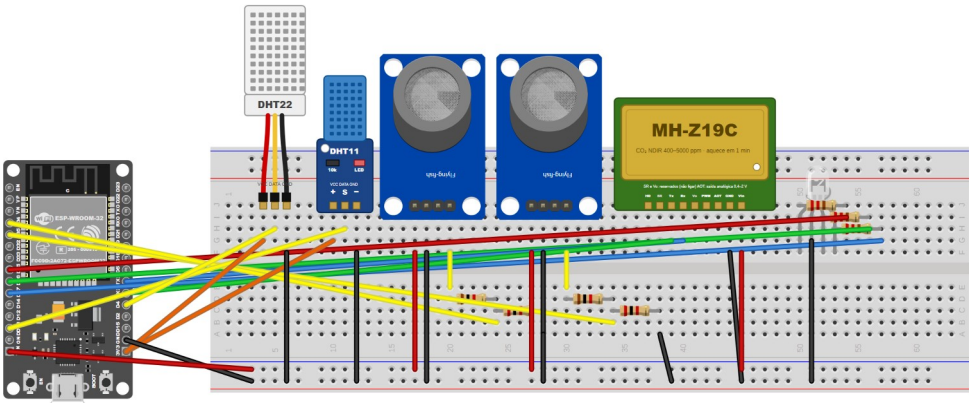


Figura 13 — Parte 6B — monitor completo no ESP32: DHTs no 3V3, MQ e MH-Z19C no VIN (5 V), divisores dos MQ na metade de baixo da protoboard, LED RGB à direita.

Passo a passo

1. **Instale a CriaTelemetria:** copie a pasta `telemetria-cria/firmware/biblioteca-cria` para a pasta `libraries` do seu sketchbook (veja em Arquivo → Preferências) e reinicie a IDE. Instale também a **PubSubClient**. No terminal: `ln -s /caminho/telemetria-cria/firmware/biblioteca-cria ~/Arduino/libraries/CriaTelemetria`.
2. Na pasta `06b_monitor_ar_esp32_telemetria`, **copie `credenciais.exemplo.h` para `credenciais.h`** e preencha: Wi-Fi da telemetria, IP do servidor, `MQTT_USUARIO` = o projeto (`ar` ou `teste`) e a senha. **O `credenciais.h` nunca vai para o GitHub** (está no `.gitignore`).
3. No sketch, confira `PROJETO` (igual ao `MQTT_USUARIO`) e escolha um `NO` **livre**: o `no1` do projeto `ar` é o monitor fixo da sala do Espaço CRIA. Dois nós com o mesmo nome se derrubam no broker.
4. Escreva os **R0** dos MQ (meça de novo com os sketches 03b e 04b, **na montagem do ESP32**: o ADC e o divisor são outros, e o R0 vale para a montagem em que foi medido).
5. Placa **DOIT ESP32 DEVKIT V1** (ou ESP32 Dev Module), grave e abra o Monitor Serial: você verá o Wi-Fi, o broker, a hora certa e cada leitura publicada.
6. Abra o painel (`http://IP_DO_SERVIDOR:3000`): o nó aparece **on-line** e os gráficos começam a se desenhar.

conferindo no terminal o que o nó publica

```
$ mosquitto_sub -h 192.168.0.10 -u ar -P SENHA -t "cria/ar/no2/#" -v
cria/ar/no2/status online
cria/ar/no2/temperatura {"valor":27.4,"unidade":"C","seq":31,"boot":3,"t":1793732797772}
cria/ar/no2/co2 {"valor":655,"unidade":"ppm","seq":33,"boot":3,"t":1793732797812}
```

06b_monitor_ar_esp32_telemetria.ino

```
/*
  06b - PROJETO: Monitor de qualidade do ar no ESP32, publicando na REDE DE TELEMETRIA DO ESPACO
CRIA
  Placa: ESP32 Dev Module / DOIT ESP32 DEVKIT V1 (30 pinos)
  Bibliotecas: CriaTelemetria (telemetria-cria/firmware/biblioteca-cria), PubSubClient, DHT
sensor library

  O que ele faz: le os 5 sensores a cada 10 s e publica no broker MQTT da escola, nos topicos do
  catalogo (telemetria-cria/mqtt/catalogo-topicos.md, secao 4.1):
     cria/ar/<no>/temperatura   C       DHT22
     cria/ar/<no>/umidade       %       DHT22
     cria/ar/<no>/co2           ppm     MH-Z19C (depois de 1 min de aquecimento)
     cria/ar/<no>/mq2           Rs/R0   MQ-2   (depois do aquecimento e com R0 medido)
     cria/ar/<no>/mq135         Rs/R0   MQ-135 (idem)
  O servidor grava no InfluxDB e o painel web mostra tudo ao vivo, sem mudar nada no servidor.
  O DHT11 fica so no Monitor Serial, comparado com o DHT22 (o catalogo nao aceita duas
  "temperatura" no mesmo no).

  Ligacoes (iguais as do firmware/no-ar, mais o DHT11 e o LED):
    DHT22 (3 fios)          vermelho -> 3V3   amarelo -> GPIO 4    preto -> GND
    DHT11 (modulo 3 pinos)  + -> 3V3          DATA (meio) -> GPIO 13   - -> GND
    MQ-2                    VCC -> VIN (5 V)  GND -> GND   AO -> 1 k -> GPIO 34 -> 2 k -> GND
    MQ-135                  VCC -> VIN (5 V)  GND -> GND   AO -> 1 k -> GPIO 35 -> 2 k -> GND
    MH-Z19C                 Vin -> VIN (5 V)  GND -> GND   TX -> GPIO 16 (RX2)   RX -> GPIO 17
(TX2)
    LED RGB catodo comum    R -> 220 R -> GPIO 25   G -> 220 R -> GPIO 26   B -> 220 R -> GPIO 27
  GPIO 21 e 22 ficam livres para o I2C, como manda o projeto. GPIO 34/35 sao do ADC1 (o ADC2 nao
  funciona com o Wi-Fi ligado).
  ALIMENTACAO: mais de 0,6 A no total -> fonte 5 V / 2 A no VIN, GND comum a todos.

  ANTES DE COMPILAR:
   1. Copie credenciais.exemplo.h (nesta pasta) para credenciais.h e preencha: Wi-Fi da
telemetria,
      IP do servidor, MQTT_USUARIO = o PROJETO abaixo ("ar" ou "teste") e a senha desse projeto.
      O credenciais.h nunca vai para o git.
   2. Escolha um NO livre (veja o painel). Dois nos com o mesmo nome se derrubam no broker.
   3. Meca o R0 dos MQ (sketches 03b e 04b, em ar limpo) e escreva abaixo. Com 0, o MQ nao e
publicado.
*/
#include <CriaTelemetria.h>
#include <DHT.h>

#if __has_include("credenciais.h")
#include "credenciais.h"
#else
#error "Falta o credenciais.h: copie credenciais.exemplo.h para credenciais.h nesta pasta e
preencha."
#endif

// ------------------------------------------------------------------- ajustes
const char* PROJETO = "ar";       // "ar" = monitor de qualidade do ar; "teste" = oficinas
const char* NO = "no2";           // o no1 do projeto "ar" e o monitor fixo da sala do Espaco
CRIA

const int PINO_DHT22 = 4, PINO_DHT11 = 13;
const int PINO_MQ2 = 34, PINO_MQ135 = 35;
const int PINO_CO2_RX = 16, PINO_CO2_TX = 17;
const int LED_R = 25, LED_G = 26, LED_B = 27;

const float DIVISOR_SUPERIOR_KOHM = 1.0;   // da saida AO ate o pino (o no-ar usa 100 k / 200 k:
e so trocar)
const float DIVISOR_INFERIOR_KOHM = 2.0;   // do pino ate o GND
const float TENSAO_DOS_MQ = 5.0;           // meca no VIN
const float RL_MQ2_KOHM = 1.0, RL_MQ135_KOHM = 1.0;   // resistor de carga do modulo ("102" = 1
k)
const float R0_MQ2_KOHM = 0.0, R0_MQ135_KOHM = 0.0;   // ar limpo, medido por voce (registre a
data)
const float LIMIAR_GAS_MQ2 = 0.5;

const int FAIXA_CO2_PPM = 5000;
const uint32_t PREAQUECIMENTO_CO2_MS = 1UL * 60 * 1000;   // MH-Z19C: 1 min
const uint32_t PREAQUECIMENTO_MQ_MS = 10UL * 60 * 1000;
const uint32_t MENOR_MV = 150, MAIOR_MV = 2450;      // faixa confiavel do ADC (11 dB)
// ------------------------------------------------------------------------
```

```
CriaNo no(PROJETO, NO);
DHT dht22(PINO_DHT22, DHT22);
DHT dht11(PINO_DHT11, DHT11);

bool co2Aquecido = false, mqAquecidos = false, gasDetectado = false;
int ultimoCO2 = -1;
uint32_t ultimoPisca = 0;
bool piscaAceso = false;

// ---------------------------------------------------------------- sensores
// Rs do MQ (kOhm) pela tensao no pino; NAN fora da faixa confiavel. Mesma conta do no-ar.
// saturado = tensao acima da faixa do ADC: gas MUITO alto (nao publica, mas e alarme).
float resistenciaDoMq(int pino, float rlKohm, bool* saturado = nullptr) {
  uint32_t soma = 0;
  for (int i = 0; i < 16; i++) soma += analogReadMilliVolts(pino);
  uint32_t mv = soma / 16;
  if (saturado) *saturado = mv > MAIOR_MV;
  if (mv < MENOR_MV || mv > MAIOR_MV) return NAN;
  float fator = (DIVISOR_SUPERIOR_KOHM + DIVISOR_INFERIOR_KOHM) / DIVISOR_INFERIOR_KOHM;
  float tensaoNoSensor = mv / 1000.0 * fator;
  if (tensaoNoSensor >= TENSAO_DOS_MQ) return NAN;
  float cargaDoDivisor = DIVISOR_SUPERIOR_KOHM + DIVISOR_INFERIOR_KOHM;
  float carga = rlKohm * cargaDoDivisor / (rlKohm + cargaDoDivisor);   // RL em paralelo com o
divisor
  return carga * (TENSAO_DOS_MQ - tensaoNoSensor) / tensaoNoSensor;
}

int lerCO2() {
  const uint8_t pedido[9] = {0xFF, 0x01, 0x86, 0x00, 0x00, 0x00, 0x00, 0x00, 0x79};
  while (Serial2.available()) Serial2.read();
  Serial2.write(pedido, sizeof(pedido));
  uint8_t r[9];
  if (Serial2.readBytes(r, sizeof(r)) != sizeof(r) || r[0] != 0xFF || r[1] != 0x86) return -1;
  uint8_t soma = 0;
  for (int i = 1; i < 8; i++) soma += r[i];
  if ((uint8_t)(0xFF - soma + 1) != r[8]) return -1;
  return r[2] * 256 + r[3];
}

// publica Rs/R0 ou, sem R0, mostra o Rs para voce calibrar
void publicarMq(const char* grandeza, const char* nome, float rs, float r0) {
  if (isnan(rs))   Serial.printf("[ar] %s: tensao fora da faixa do ADC, nao publicado\n", nome);
  else if (r0 > 0) no.publicar(grandeza, rs / r0, "Rs/R0");
  else             Serial.printf("[ar] %s: Rs = %.2f kOhm (R0 ainda nao calibrado: nada
publicado)\n", nome, rs);
}

// ---------------------------------------------------------------- LED
void cor(bool r, bool g, bool b) { digitalWrite(LED_R, r); digitalWrite(LED_G, g);
digitalWrite(LED_B, b); }

void atualizarLed() {
  if (millis() - ultimoPisca < 300) return;
  ultimoPisca = millis();
  piscaAceso = !piscaAceso;
  if (gasDetectado)        cor(piscaAceso, 0, 0);       // gas: vermelho piscando
  else if (!no.conectado()) cor(piscaAceso, 0, piscaAceso);   // sem broker: roxo piscando
(leituras vao para a fila)
  else if (!co2Aquecido)   cor(0, 0, 1);                // aquecendo: azul
  else if (ultimoCO2 < 0)  cor(0, 0, piscaAceso);       // MH-Z19C sem resposta: azul piscando
  else if (ultimoCO2 <= 800)  cor(0, 1, 0);
  else if (ultimoCO2 <= 1000) cor(1, 1, 0);
  else                        cor(1, 0, 0);
}

// ---------------------------------------------------------------- programa
void setup() {
  Serial.begin(115200);
  pinMode(LED_R, OUTPUT); pinMode(LED_G, OUTPUT); pinMode(LED_B, OUTPUT);
  cor(1, 1, 1); delay(300); cor(0, 0, 0);
  analogSetPinAttenuation(PINO_MQ2, ADC_11db);
  analogSetPinAttenuation(PINO_MQ135, ADC_11db);
  Serial2.begin(9600, SERIAL_8N1, PINO_CO2_RX, PINO_CO2_TX);
  Serial2.setTimeout(200);
```

```
  dht22.begin();
  dht11.begin();
  no.mostrarPublicacoes(true);          // mostra no Monitor Serial cada leitura publicada
  no.iniciar(WIFI_SSID, WIFI_SENHA, MQTT_HOST, MQTT_PORTA, MQTT_USUARIO, MQTT_SENHA);
}

void loop() {
  no.manter();                          // Wi-Fi, broker, hora certa e fila: sem delay() longo!
  atualizarLed();
  if (!no.aCada(10000)) return;         // a cada 10 s, e so depois de acertar a hora

  if (millis() >= PREAQUECIMENTO_CO2_MS) co2Aquecido = true;
  if (millis() >= PREAQUECIMENTO_MQ_MS) mqAquecidos = true;

  // DHT22 -> telemetria (NaN e recusado pela biblioteca: leitura com falha nao e publicada)
  float t22 = dht22.readTemperature(), h22 = dht22.readHumidity();
  no.publicar("temperatura", t22, "C");
  no.publicar("umidade", h22, "%");

  // DHT11 -> so comparacao
  float t11 = dht11.readTemperature(), h11 = dht11.readHumidity();
  Serial.printf("[ar] DHT11 %.1f C %.0f %%  x  DHT22 %.1f C %.1f %%  (diferenca %.1f C, %.0f %%)
\n",
                t11, h11, t22, h22, t11 - t22, h11 - h22);

  if (co2Aquecido) {
    int co2 = lerCO2();
    if (co2 > 0 && co2 <= FAIXA_CO2_PPM) { ultimoCO2 = co2; no.publicar("co2", co2, "ppm"); }
    else { ultimoCO2 = -1; Serial.printf("[ar] MH-Z19C sem resposta ou fora da faixa (%d)\n",
co2); }
  }

  if (mqAquecidos) {
    bool saturado = false;
    float rs2 = resistenciaDoMq(PINO_MQ2, RL_MQ2_KOHM, &saturado);
    gasDetectado = saturado || (R0_MQ2_KOHM > 0 && !isnan(rs2) && rs2 / R0_MQ2_KOHM <
LIMIAR_GAS_MQ2);
    publicarMq("mq2", "MQ-2", rs2, R0_MQ2_KOHM);
    publicarMq("mq135", "MQ-135", resistenciaDoMq(PINO_MQ135, RL_MQ135_KOHM), R0_MQ135_KOHM);
  } else {
    Serial.printf("[ar] pre-aquecendo os MQ: faltam %lu s\n", (unsigned long)
((PREAQUECIMENTO_MQ_MS - millis()) / 1000));
  }
}
```

Entenda o código

- `CriaNo no(PROJETO, NO)` cria o nó; `no.iniciar(...)` liga Wi-Fi, MQTT e relógio. **Nada de `delay()` longo** no `loop()`: o `no.manter()` precisa rodar sempre.
- `no.aCada(10000)` fica verdadeiro a cada 10 s, e só depois que o relógio foi acertado: a própria biblioteca recusa publicar sem hora certa (e o servidor recusaria uma hora errada).
- `no.publicar()` **recusa sozinho** valores NaN (o DHT22 desligado). Os outros sinais de falha (MH-Z19C sem resposta, MQ fora da faixa do ADC) o nosso código confere antes: **leitura com falha não é publicada**.
- Se o Wi-Fi cair, as leituras vão para a **fila na flash** e são reenviadas com `"fila":1` quando a conexão voltar — teste desligando o roteador por 1 minuto.

Problemas comuns

Sintoma	Causa provável	Solução
DHT: só “FALHA”	dado no pino errado; VCC/GND trocados; leitura rápida demais	confira a serigrafia e o pino no código; intervalo ≥ 2 s
DHT11 e DHT22 discordam 2 °C	exatidão do DHT11 (±2 °C)	normal; use o DHT22 como referência
MQ: Rs/R0 muito diferente de 1 no ar limpo	R0 medido antes do aquecimento ou em ar com gás	aqueça 10 min, meça o R0 de novo com a janela aberta
MQ no ESP32: “fora da faixa do ADC”	AO sem divisor, fio solto, ou RL de 1 kΩ deixando a tensão muito baixa	confira o divisor; meça AO com multímetro; RL maior (10 kΩ) se o pino ficar < 0,15 V
MH-Z19C: sem resposta	Tx/Rx trocados; fio em SR/Vo; falta do divisor no Uno; Vin sem 5 V	troque Tx↔Rx; confira os nomes e 5 V no Vin
MH-Z19C: sempre 400/410/500	ainda aquecendo	espere 1 min
MH-Z19C: valor baixo demais (< 380)	zero calibrado em ambiente com CO_2 alto, ou ABC numa sala sempre fechada	recalibre ao ar livre (`z`), desligue a ABC (`d`)
ESP32 reinicia sozinho (brownout)	fonte fraca	fonte 5 V/2 A no VIN
Nó não aparece no painel	credenciais, projeto ≠ usuário MQTT, nó repetido, servidor fora	leia o Monitor Serial (prefixo `[cria]`); confira com `mosquitto_sub`
Gateway: “porta ocupada”	Monitor Serial aberto	feche o Monitor Serial

Desafios

1. **Comparação DHT11 × DHT22:** registre 1 h de leituras dos dois (exporte o CSV do painel) e calcule a diferença média e o desvio. O DHT11 cabe nos ±2 °C?
2. **MQ-135 × MH-Z19C:** faça um gráfico de dispersão do “CO_2 estimado” do MQ-135 contra o CO_2 do MH-Z19C durante uma aula. Há correlação? E com álcool por perto?
3. **Buzzer de ventilação:** acrescente o buzzer passivo e toque 3 bipes quando o CO_2 passar de 1000 ppm (só uma vez, até voltar abaixo de 900: **histerese**).
4. **Contagem de pessoas:** com a sala fechada, o CO_2 sobe mais rápido quanto mais gente há. Meça a inclinação (ppm/min) com 5, 10 e 20 pessoas.
5. **Leitura pelo PWM:** o MH-Z19C também informa o CO_2 pela largura do pulso no pino PWM (ciclo de 1004 ms; C = 2000 × (TH − 2 ms)/(T − 4 ms) na faixa de 2000 ppm). Use `pulseIn()` e compare com a serial.
6. **AOT pelo ADS1115:** ligue a saída analógica AOT (0,4–2 V) do MH-Z19C no A1 do ADS1115 e levante a reta tensão × ppm comparando com a serial.
7. **Alarme de gás independente:** use o DO do MQ-2 (com o trimpot ajustado) para acender um LED vermelho mesmo se o programa travar.

8. **Painel próprio:** use a rota `/api/historico` do servidor da telemetria para desenhar, num script Python, o CO_2 das últimas 24 h.

Referências

- Winsen — *Infrared CO_2 Sensor Module MH-Z19C, User's Manual* v1.2 (2021): pinos, faixas, 1 min de aquecimento, comandos 0x86/0x79, zero por HD.
- Hanwei — folhas de dados MQ-2 e MQ-135 (curvas de sensibilidade).
- Aosong — folhas de dados DHT11 e AM2302/DHT22.
- ANVISA, Resolução RE nº 9/2003 — padrões de qualidade do ar interior (CO_2 ≤ 1000 ppm).
- Winsen — *MH-Z19B User's Manual* v1.6 (comandos 0x87/0x88/0x99 e PWM, que o MH-Z19C mantém).
- Texas Instruments — *ADS111x* (SBAS444).
- Rede de Telemetria do Espaço CRIA — `telemetria-cria/mqtt/catalogo-topicos.md`, `firmware/biblioteca-cria/README.md`, `gateway/README.md`.